Documentação — Fase 2 Pós-Tech

Índice

1. Relatório 1 — Otimização de modelos de diagnóstico com algoritmo genético e LLM (PCOS)

Relatório 1 — Otimização de modelos de diagnóstico (PCOS)

1. Link do Git

https://github.com/icaroamerico/pos-tech-fiap-ia-dev/tree/main/fase2

2. Caminho do README.md

https://github.com/icaroamerico/pos-tech-fiap-ia-dev/blob/main/fase2/README.md

3. Caminho do Dockerfile

N/A

4. Caminho do Dataset

Dataset original: https://github.com/icaroamerico/pos-tech-fiap-ia-dev/blob/main/fase1/base_dados/PCOS_infertility.csv
https://github.com/icaroamerico/pos-tech-fiap-ia-dev/blob/main/fase1/base_dados/PCOS_data_without_infertility.xlsx

Dataset tratado (usado pelo AG):
https://github.com/icaroamerico/pos-tech-fiap-ia-dev/blob/main/fase2/base_dados_tratada/PCOS_unificado.csv

5. Vídeo de demonstração

N/A

6. Link do vídeo

https://www.youtube.com/watch?v=1qFEbze5avE

7. Resultados obtidos + Relatório técnico

Otimização de Modelos de Diagnóstico com Algoritmo Genético e LLM

Aluno: Icaro Minervini Americo — RM376755
Turma: 10IADT · POS TECH FIAP, *IA para Devs*
Data: 18/10/2026
Repositório: https://github.com/icaroamerico/pos-tech-fiap-ia-dev/tree/main/fase2
Vídeo de demonstração: https://www.youtube.com/watch?v=1qFEbze5avE
Projeto escolhido: Projeto 1 — *Otimização de Modelos de Diagnóstico*

Sumário

1. Contexto: o que a Fase 1 entregou e o que ficou em aberto
2. Implementação do algoritmo genético
3. Os três experimentos: configurações, hipóteses e resultados
4. Comparativo original vs. otimizado e o baseline de busca aleatória
5. Integração com LLM para interpretação dos resultados
6. Arquitetura, monitoramento e logging
7. Desafios enfrentados e soluções implementadas
8. Ética e limitações
9. Conclusão e ponte para o Módulo 3

Anexos: onde está cada artefato · Anexo A — os 3 templates de prompt, texto integral

Figuras (geradas pelos notebooks `01` e `02` a partir dos artefatos em `arquivos_saida/arquivos_analiticos/historico_execucoes_ag/`; ficam em `documentacao/imagens/`)

1. Convergência e diversidade de uma execução E1 (Notebook 01) — §2.7
2. Convergência do AG em E1, E2 e E3, média ± desvio sobre 3 sementes — §3.2
3. Diversidade da população em E1, E2 e E3 — §3.3
4. Etapa D: ablação torneio vs. roleta — §3.4
5. Distribuição das famílias na população, E1/semente 42 — §3.6
6. Original vs. otimizado nos 5 modelos, conjunto de teste — §4.1

> **Convenção deste relatório.** Cada decisão técnica é apresentada na mesma ordem: **o que** foi feito → **por que** essa escolha e não outra → **como isso afeta o resultado** → **como cheguei a essa conclusão** (qual evidência, qual conta, qual referência). É o mesmo padrão dos notebooks da Fase 1 e dos ADRs deste repositório.

> **Procedência dos números.** Todas as tabelas deste relatório foram geradas a partir dos artefatos em `arquivos_saida/arquivos_analiticos/historico_execucoes_ag/` pelas funções de `src/experimentos/analise.py` — não foram transcritas à mão, exceto onde indicado (a tabela AG × busca aleatória do §4.2, cuja procedência está descrita lá). A razão está no §7, achado nº 10: transcrição manual foi exatamente a origem de cinco células erradas na documentação anterior.

---

1. Contexto

1.1 O que a Fase 1 entregou

A Fase 1 produziu um pipeline de classificação de Síndrome dos Ovários Policísticos (SOP / PCOS) sobre dados tabulares clínicos e hormonais: base final de **541 linhas × 49 colunas**, split 80/20 estratificado com `SEED=42` → **432 linhas de treino / 109 de teste** (36 positivas no teste). Cinco famílias de classificador clássico foram treinadas e avaliadas:

Modelo	Acurácia	Precisão	Recall	F1	AUC
Random Forest	**0,9174**	**0,9355**	0,8056	**0,8657**	**0,9365**
SVM (RBF)	0,8807	0,8108	0,8333	0,8219	0,9315
Regressão Logística	0,8532	0,7381	**0,8611**	0,7949	0,9277
Árvore de Decisão	0,8624	0,8000	0,7778	0,7887	0,8809

Modelo	Acurácia	Precisão	Recall	F1	AUC
KNN (k=5)	0,8624	0,8889	0,6667	0,7619	0,9007

Como cheguei a estes números na Fase 2: os scripts `src/modelagem/treinamento.py` e `src/run_ml.py` são **cópias da Fase 1** (ADR 009) — o `run_ml.py` byte a byte, o `treinamento.py` com uma linha alterada, o `REPORTS_DIR`, e nada mais —, e `python -m src.run_ml` reproduz a tabela acima **até a quarta casa decimal**. Esse passo é o primeiro do `README.md` por um motivo: sem baseline reproduzido e congelado, qualquer ganho reportado depois seria ficção.

1.2 O que ficou em aberto — e por que a Fase 2 é o passo seguinte

O notebook da Fase 1 encerra listando **"ajuste de hiperparâmetros com GridSearchCV / validação cruzada"** como próximo passo não executado. A Fase 2 é literalmente esse passo, com algoritmo genético em vez de grid.

> ⚠️ **Correção a uma premissa do plano de implementação.** O plano (§1.1) afirmava que a Fase 1 usou "hiperparâmetros default do scikit-learn". **Não usou.** Auditando `treinamento.py` encontrei `class_weight="balanced"` em todos os modelos, `n_estimators=200` e `max_depth=8` no Random Forest, `max_depth=5` na árvore e `max_iter=2000` na regressão logística. Isso muda a leitura do trabalho em duas direções opostas, e as duas vão declaradas: **fortalece** o *hotstart* (§2.3), porque o ponto de partida é uma configuração conscientemente escolhida e não um default arbitrário; e **eleva a barra**, porque o AG precisa superar algo já razoável. A segunda consequência é a mais importante para interpretar o §4.

Medindo a configuração da Fase 1 com o fitness da Fase 2 (score composto em validação cruzada k=5 sobre as 432 linhas de treino — ver §2.4), o baseline a bater é:

Família	Fitness CV da config. da Fase 1	F1	Recall	AUC
SVM (RBF)	**0,8391**	0,8118	0,8160	0,9420
Regressão Logística	0,8337	0,7973	0,8300	0,9304
Random Forest	0,8254	0,8170	0,7522	**0,9562**
KNN	0,7232	0,7149	0,6177	0,9021
Árvore de Decisão	0,7123	0,6826	0,7165	0,7799

Observação que já antecipa o §4: em CV, o Random Forest — vencedor no holdout da Fase 1 — é apenas o **terceiro**. A ordenação das famílias depende do recorte de dados e da métrica, e isso é um aviso sobre o tamanho da base (541 registros), não sobre os modelos.

1.3 Escopo declarado

Item	Decisão	Onde está justificada
O AG roda somente no PCOS	A CNN de raio-X da Fase 1 está integralmente fora	ADR 001
O plural "*os modelos de diagnóstico*" do enunciado	Atendido pelas **cinco famílias** do PCOS, com comparação individual antes/depois (§4.1)	ADR 001
Nuvem / IaC	**Fora de escopo** — o enunciado declara a nuvem opcional	—
Implantação, autoscaling real	Não implementados	§6.3

Por que a CNN ficou fora, em uma linha: um AG sobre a CNN com fidelidade plena custaria ~27 horas de treino; com proxy de baixa fidelidade (3 épocas, 30% dos dados) custaria ~25 min, mas **mediria a coisa errada** — arquiteturas com BatchNorm e dropout alto convergem mais devagar e seriam sistematicamente penalizadas, levando o AG a escolher a arquitetura errada pelo motivo errado. O orçamento foi gasto, em vez disso, em profundidade sobre o alvo

tabular: 3 experimentos × 3 sementes, ablação de operador de seleção, e baseline de busca aleatória — tudo que o §3 e o §4 reportam.

---

2. Implementação do algoritmo genético

> Enunciado, requisito 1: *definir codificação adequada; implementar seleção, cruzamento e mutação; definir fitness baseada em métricas de desempenho; comparar otimizado vs. original; realizar ao menos 3 experimentos com diferentes configurações do AG.*

O AG foi **implementado do zero** (numpy + Python padrão), sem DEAP ou PyGAD (ADR 004). **Por que:** o trabalho pede que eu descreva **meus** operadores, e usar biblioteca pronta esvaziaria exatamente a parte avaliada. O fluxo segue o laço evolutivo canônico: inicialização, avaliação, seleção, crossover, mutação, elitismo e critério de parada.

Tamanho da implementação: 3.763 linhas em `src/` (17 módulos), 1.679 linhas em `tests/` (**221 testes, todos passando** — `pytest tests/ -q` em 105 s).

2.1 Codificação dos indivíduos — híbrida

São quatro as codificações clássicas de indivíduo em AG: binária, real, combinatória e híbrida. A híbrida combina tipos de dados diferentes — cada trecho representa um aspecto distinto da solução, com restrições próprias.

Hiperparâmetros são heterogêneos por natureza: `C` do SVM é real e vive em escala logarítmica, `n_estimators` é inteiro, `criterion` é categórico. **Forçar tudo em codificação binária exigiria decodificação artificial e jogaria fora a estrutura do problema.** Daí a codificação híbrida, com o tipo de cada gene declarado num `SCHEMA` (ADR 002).

Um indivíduo é um dicionário com **24 genes** distribuídos em cinco blocos, mais o gene de família:

```
{
  "familia": "svm_rbf",                      # gene 0 — escolhe o modelo
  "random_forest": {...7 genes...},          # latente (não expresso)
  "svm_rbf": {"C": 0.0, "gamma": -1.66, ...}, # ATIVO — estes afetam o fitness
  "logistic_regression": {...4 genes...},    # latente
  "decision_tree": {...6 genes...},          # latente
  "knn": {...4 genes...},                    # latente
}
```

Os seis tipos de gene, e o que cada tipo determina (amostragem, crossover, mutação, clamp):

Tipo	Significado	Exemplo
`int`	inteiro uniforme	`n_estimators: 50–300`
`int_odd`	inteiro **ímpar** — evita empate em classificação binária	`n_neighbors: 1–31`
`int_or_none`	inteiro ou `None` (irrestrito)	`max_depth: 2–20` ou `None`
`real`	real uniforme	`ccp_alpha: 0–0,05`
`real_log`	guarda **`log10(valor)`** internamente	`C: 10⁻² – 10³`
`categorical`	um de uma lista (string, número, `None` ou tupla)	`criterion: gini | entropy`

Por que `C` e `gamma` em escala logarítmica. A sensibilidade do SVM a esses parâmetros é multiplicativa, não aditiva. Mutação gaussiana em escala linear sobre `[1e-2, 1e3]` quase nunca sortearia valores abaixo de 1 — a busca ficaria cega à metade do espaço útil. Os genes guardam `log10(valor)`, mutam em escala log e são exponenciados na decodificação. Há um efeito colateral útil: **o crossover aritmético em espaço log equivale à média geométrica no espaço real**, que é a interpolação correta para esses parâmetros.

Por que carregar genes latentes em vez de cromossomo de tamanho variável. Cromossomo de tamanho fixo mantém crossover e mutação triviais e sempre válidos. Genes latentes funcionam como reservatório de diversidade: se

o gene de família mutar de SVM para Random Forest, os hiperparâmetros de RF que o indivíduo carregava adormecidos voltam a ser expressos, em vez de serem sorteados do zero. É o paralelo direto de material genético recessivo.

Como isso afeta o resultado — e é a consequência mais forte da decisão: o gene 0 transforma o AG em algo mais do que busca de hiperparâmetros. Ele faz **seleção de modelo e ajuste de hiperparâmetros simultaneamente**: a pressão seletiva decide qual família merece orçamento de busca, em vez de eu decidir a priori. Isso gera uma pergunta de pesquisa que o §3.6 responde com dados — *o AG converge para Random Forest, como a Fase 1 sugeriria, ou outra família ultrapassa quando todas são bem ajustadas?*

Espaço de busca por família

Família	Gene	Tipo	Faixa / valores
Random Forest (7 genes)	`n_estimators`	`int`	50–300
	`max_depth`	`int_or_none`	2–20 ou `None`
	`min_samples_split`	`int`	2–20
	`min_samples_leaf`	`int`	1–10
	`max_features`	`categorical`	`sqrt`, `log2`, 0,3, 0,5, 0,8
	`criterion`	`categorical`	`gini`, `entropy`
	`class_weight`	`categorical`	`None`, `balanced`
SVM (RBF) (3 genes)	`C`	`real_log`	10^{-2} – 10^{3}
	`gamma`	`real_log`	10^{-4} – 10^{1}
	`class_weight`	`categorical`	`None`, `balanced`
Regressão Logística (4 genes)	`C`	`real_log`	10^{-3} – 10^{2}
	`penalty_solver`	`categorical` **pareado**	`l2/lbfgs`, `l1/liblinear`, `elasticnet/saga`
	`l1_ratio`	`real`	0–1 (só expresso com `elasticnet`)
	`class_weight`	`categorical`	`None`, `balanced`
Árvore de Decisão (6 genes)	`max_depth`	`int_or_none`	2–20 ou `None`
	`min_samples_split`	`int`	2–20
	`min_samples_leaf`	`int`	1–10
	`criterion`	`categorical`	`gini`, `entropy`
	`ccp_alpha`	`real`	0–0,05
	`class_weight`	`categorical`	`None`, `balanced`
KNN (4 genes)	`n_neighbors`	`int_odd`	1–31
	`weights`	`categorical`	`uniform`, `distance`
	`metric`	`categorical`	`euclidean`, `manhattan`, `minkowski`
	`p`	`int`	1–5 (só expresso com `minkowski`)

Teto de `n_estimators` em 300: acima disso o ganho satura e o custo por avaliação cresce linearmente, atrasando as gerações — é custo direto de eficiência computacional. **Faixas de profundidade apertadas:** com 432 linhas de treino, profundidade alta memoriza.

Validade por construção — três mecanismos, nenhum de reparo

1. `clamp_individuo` é chamado **após cada operador**, forçando todo gene de toda família à sua faixa válida;

2. `penalty` / `solver` são herdados como **par atômico** — nunca recombinados entre si, porque nem toda combinação é aceita pelo scikit-learn;
3. `l1_ratio` e `p` existem como genes, mas só são **expressos** sob condição (`elasticnet` / `minkowski`).

Consequência concreta: não existe `try/except` em volta do treino em nenhum ponto do projeto. Isso é decisão, não descuido. Um `try/except` mascararia indivíduos inválidos como "fitness zero", e o AG aprenderia a evitar regiões **válidas** do espaço de busca por motivo errado — um erro que não dá erro. **Como provei que a garantia vale:** `test_cromossomo.py` instancia **300 indivíduos aleatórios no scikit-learn de verdade** e falha se qualquer um for rejeitado.

2.2 Operadores genéticos

Seleção — torneio, com elitismo (desvio deliberado do material)

A referência clássica é a **seleção probabilística tipo roleta**: seleção aleatória ponderada pelo valor de fitness, escrita como `probability = 1 / np.array(population_fitness)` em problemas de minimização, como o TSP.

Usei torneio, e este é um desvio consciente do material, que precisa de justificativa (ADR 005). A razão é a escala do fitness:

1. **A roleta depende da escala absoluta do fitness; o torneio não.** No TSP clássico o fitness é distância, varia de centenas a milhares de pixels, e `1/distância` gera pesos genuinamente distintos. **Aqui o fitness é métrica de classificação confinada a ~[0,75; 0,92]** — razão melhor/pior de ~1,2. A roleta daria a praticamente todos a mesma probabilidade: pressão seletiva quase nula.
2. **O torneio só compara, nunca pondera** — é invariante a transformações monotônicas do fitness, logo imune ao problema acima.
3. **Pressão seletiva ajustável** via `k`, independente da escala — é o que permite o experimento E3 (§3) variar pressão seletiva sem mexer no fitness.
4. Custo O(1) por seleção, contra O(n) da roleta.

> **Para não deixar isso como alegação teórica, implementei os dois operadores e rodei a ablação** (Etapa D, §3.4). A previsão foi declarada antes, versionada em `src/experimentos/configuracoes.py`, e o resultado está no §3.4 — incluindo a parte em que a previsão errou o sinal da observável.

⚠️ **Detalhe de implementação que virou bug e foi corrigido:** o torneio sorteia **sem reposição** (`rng.choice(..., replace=False)`). Com reposição, um torneio `k=2` sobre população pequena podia sortear o mesmo indivíduo duas vezes e nunca comparar um segundo candidato — pressão seletiva inconsistente justamente em E3, o experimento que depende dela. Ver §7, achado nº 4.

Elitismo: os `n_elite = 2` de 30 (~6,7%) melhores passam **intactos** para a próxima geração. **Por que:** sem elitismo, crossover e mutação podem destruir o melhor indivíduo já encontrado; "risco de destruir boas soluções" é o custo conhecido da mutação. Elitismo garante que **a curva de melhor-fitness é monotonicamente não decrescente**, o que torna o gráfico de convergência interpretável. Verificado por invariante em `test_motor.py` ("melhor fitness nunca piora").

Cruzamento — despachado por tipo de gene, taxa 0,8

O método de crossover depende do tipo de codificação, porque o resultado do cruzamento precisa ser uma solução válida. Numa codificação híbrida, cada tipo de dado pede um tipo específico de cruzamento.

Tipo de gene	Operador	Fonte / por quê
`real`, `real_log`, `int`	**Arithmetic crossover**: `filho = α·pai1 + (1−α)·pai2`, α~U(0,1); inteiros com arredondamento e *clamp*	Operador padrão de codificação real. Interpola entre dois bons indivíduos. Em `real_log`, a média aritmética em log = **média geométrica** no espaço real

Tipo de gene	Operador	Fonte / por quê
`categorical`, booleano, gene de família	**Uniform crossover**: por gene, sorteia de qual pai herdar	Operador padrão para genes categóricos. Média de `gini` e `entropy` não significa nada; a única operação válida é escolher um dos dois

Com probabilidade `1 − taxa_crossover` os filhos são cópias dos pais. Tudo termina em `clamp_individuo`. **Aplicar o operador errado produziria indivíduo inválido** — exatamente o critério de validade que o operador precisa preservar.

Mutação — probabilidade e intensidade separadas

A mutação tem dois parâmetros distintos: **probabilidade de mutação** (chance de mutar) e **intensidade da mutação** (quão forte). **Implementei os dois separadamente**, porque é o que permite montar os experimentos do §3 variando um sem o outro.

Tipo de gene	Operador	Intensidade
`real`, `real_log`, `int`	**Mutação gaussiana**: `gene += N(0, σ)`, σ = intensidade × amplitude da faixa	σ ∈ {0,05; 0,10; 0,25} nos experimentos
`categorical`	Resorteio uniforme entre os valores válidos **≠ atual**	—
`int_or_none`	Gaussiana, ou 30% de chance de voltar a `None`	—

Na mutação gaussiana, quanto maior a intensidade, maior o intervalo da distribuição e, portanto, maior a chance de uma mutação forte. O trade-off:

Mais mutação	Menos mutação
Mais exploração	Menor exploração
Menos aproveitamento	Maior aproveitamento
Mais diversidade genética	Risco de convergência prematura
Risco de destruir boas soluções	Maior chance de aperfeiçoar soluções já boas

O elitismo existe como apólice de seguro contra a última linha da coluna da esquerda.

Duas correções que vieram de auditoria (ADR 006) — ambas mudaram o significado dos parâmetros dos experimentos, e por isso estão aqui e não só no §7:

1. **`prob_troca_familia` separada do resto** (0,05). Trocar de família é mudança de natureza diferente de ajustar um hiperparâmetro: descarta todo refinamento acumulado. Com a taxa unificada, a chance de uma linhagem manter sua família por 15 gerações caía a **8,7%** com `p=0,15` e **0,1%** com `p=0,35` — a escolha do modelo virava passeio aleatório, e o gene 0 deixava de significar o que o §2.1 diz que significa. Com 0,05, sobe a **46%**.
2. **Normalização por família.** As famílias têm números diferentes de genes (3 no SVM, 7 no RF), mas **só os da família ativa afetam o fitness**. Sem normalizar, um RF sofria **2,3× mais** mutação efetiva que um SVM à mesma taxa nominal — ou seja, "prob_mutacao = 0,15" significava coisas diferentes para indivíduos diferentes. A probabilidade por gene passou a ser escalada para igualar o número *esperado* de mutações nos genes expressos, com teste de simetria e contraprova em `test_operadores.py`.

2.3 Inicialização da população — aleatória + hotstart

O contraste entre os dois métodos é conhecido: o hotstart parte de uma estimativa inicial mais informada, derivada de resultados anteriores, o que permite convergência mais rápida. **A Fase 1 é exatamente esse resultado anterior.**

- **~90% da população: aleatória** dentro das faixas, para exploração global;
- **~10% da população: hotstart** — indivíduos codificando os hiperparâmetros que a Fase 1 **realmente usou** (§1.2).

Por que não 100% hotstart: inicialização informada demais pode levar a convergência prematura em ótimo local — 100% hotstart colocaria toda a população na vizinhança de um único ponto.

> ⚠️ **Limitação do hotstart, declarada.** `n_hotstart = round(tamanho × 0,1)` e as famílias são atribuídas ciclicamente, então o hotstart **não cobre as cinco famílias em todos os experimentos**: E1 (pop 30 → 3 hotstarts) cobre RF/SVM/LR; E2 (pop 60 → 6) cobre as cinco; E3 (pop 15 → 2) cobre RF/SVM. Na Etapa A, em que a família é fixada, o motor sobrescreve o gene de família **depois** da inicialização — então em `A_logistic_regression`, `A_decision_tree` e `A_knn` **nenhum** indivíduo carrega a configuração da Fase 1. A garantia que o plano enunciava ("o melhor fitness da geração 0 é, por construção, ≥ o da Fase 1") **vale empiricamente nas cinco famílias, mas por sorte, não por construção** — os indivíduos aleatórios da geração 0 superaram o hotstart em todas elas (coluna `gen0` da tabela do §4.1). Detalhes no §7, achado nº 9.

2.4 Função fitness

Esta é a decisão mais carregada de significado do projeto, porque **define o que "melhor modelo" quer dizer**.

```
fitness(indivíduo) = média_k( 0,5 · F1 + 0,3 · recall + 0,2 · AUC )
                     em StratifiedKFold(k=5, shuffle=True, random_state=42)
                     aplicado SOMENTE às 432 linhas de treino
```

Por que não acurácia

A função de fitness é específica de cada problema e reflete os objetivos e critérios de desempenho desejados. O objetivo aqui é **diagnóstico médico**, e em diagnóstico **falso negativo e falso positivo não custam a mesma coisa**: dizer "você não tem SOP" a quem tem significa não tratar; o inverso significa um exame a mais. Acurácia trata os dois como equivalentes. **Acurácia é a métrica errada para este problema.**

Como cheguei aos pesos — a evidência está na tabela da Fase 1

O Random Forest vencedor da Fase 1 teve precisão 0,9355 mas **recall 0,8056 — sua métrica mais fraca**. Ou seja: o modelo que a Fase 1 escolheu erra justamente no tipo de erro que mais importa clinicamente — ~19% dos casos positivos passam batido. Daí os pesos:

- **F1 (0,5)** como objetivo balanceado principal;
- **recall com peso extra explícito (0,3)**, para empurrar o AG na direção da fraqueza conhecida do baseline;
- **AUC (0,2)** para premiar separabilidade independente de limiar, estabilizando o fitness quando dois indivíduos empatam em F1.

Como isso afeta o modelo, e a predição verificável que nasceu daí: a pressão de recall deveria fazer o alelo `class_weight='balanced'` se fixar na população ao longo das gerações. Essa predição é **medida por geração** e o resultado está no §3.5 — e é mais interessante do que a predição original supunha.

Por que validação cruzada e não um único split de validação

Com 432 linhas de treino, um único split de validação teria ~86 linhas — variância altíssima. O AG, otimizando centenas de vezes contra um alvo tão ruidoso, **otimizaria o ruído daquele split específico**. `StratifiedKFold` com k=5 torna o fitness muito mais estável, ao custo de 5 treinos por avaliação — custo pagável porque o alvo é tabular e rápido (~0,3 s por ajuste). Num alvo caro, o k=5 seria o primeiro item a cair, e a qualidade do fitness cairia com ele.

Por que as mesmas 5 dobras para todos os indivíduos. Reembaralhar introduziria ruído de amostragem como fator de confusão: dois indivíduos idênticos receberiam fitness diferentes, e o cache do §6.2 deixaria de ser correto. *Limitação declarada:* fixar as dobras permite que o AG se ajuste àquela partição específica — e o §4.2 mostra o preço disso.

A penalidade de custo do plano — não implementada, e declarado

> ⚠️ O plano de implementação (§4.3) previa um termo `− 0,01 · tempo_de_treino_normalizado` no fitness, como desempate de parcimônia entre indivíduos estatisticamente equivalentes, e o justificava pelo "melhorar a precisão

> **e eficiência**" do enunciado. **Esse termo não foi implementado.** A fórmula em `src/fitness/fitness_pcos.py` é a soma ponderada pura.
>
> **Consequência real:** nenhum resultado deste relatório é afetado na direção da qualidade — o peso previsto era 1%, menor que o desvio padrão entre sementes (0,0039 no E1). Mas a eficiência deixou de ser objetivo do fitness e passou a ser tratada **só** na camada de engenharia (cache e paralelismo, §6.2), que é onde ela de fato foi medida. O requisito de eficiência continua atendido; o mecanismo é outro, e está dito aqui em vez de omitido.

Reprodutibilidade

`random_state=SEED` em todo lugar: dobras do CV e todos os modelos (exceto KNN, que não tem componente estocástico). **Fitness determinístico dado o mesmo indivíduo** — condição necessária para o cache do §6.2 e verificada por teste de determinismo em `test_fitness.py`.

2.5 A regra que organiza o projeto: o fitness nunca toca o conjunto de teste

> 🔴 **Fitness = CV sobre as 432 linhas de treino. As 109 linhas de teste são tocadas exatamente uma vez, no fim, para reportar o número final.**

Por que isso é a regra nº 1 (ADR 003): um AG que otimiza contra a métrica de teste é vazamento de dados puro. Produziria um número bonito e sem valor nenhum — o AG é um otimizador agressivo e, com centenas de avaliações, *vai* encontrar a configuração que explora o ruído daquelas 109 linhas específicas. Era o erro mais provável de todo o projeto.

Como a garantia é estrutural, não convencional:

```
@dataclass(frozen=True)
class DadosTreino:
    X_treino: pd.DataFrame      # 432 linhas
    y_treino: pd.Series
    colunas_numericas: list[str]
    colunas_categoricas: list[str]
```

Não existe campo `X_teste` / `y_teste` nesta classe. Não há atributo para um bug futuro acessar, mesmo que alguém tente. `carregar_dados_treino()` chama `dividir_dados()` do script da Fase 1 e descarta as variáveis de teste — nomeadas `_X_teste_NAO_USAR` / `_y_teste_NAO_USAR` para deixar isso explícito a quem ler o código. No caminho do AG, o único módulo autorizado a ler o holdout é `src/fitness/avaliacao_final.py`: está em arquivo separado de propósito e **não é importado por nenhum código do AG**.

Há um segundo leitor do holdout no repositório, e ele é intencional. `src/llm/rodar_avaliacao.py` chama `dividir_dados()` para montar os contextos dos 15 pacientes que alimentam a camada LLM (§5). Esse código roda **depois** de a busca terminar, lê o melhor indivíduo já congelado em `best_individual.json` e não devolve nenhum número ao AG. Ou seja: o holdout nunca entra em nenhuma decisão de seleção — nem direta, nem por ajuste manual guiado pelo resultado.

Como provei que não houve vazamento: dois testes substituem o conjunto de teste por um objeto que lança exceção em **qualquer** acesso — atributo, `len()`, indexação, iteração — um para uma chamada isolada de fitness, outro para **o laço evolutivo completo**. O teste passar significa que o holdout genuinamente nunca foi tocado, não apenas que não deveria ter sido.

2.6 Critérios de parada

Combinação de três, o primeiro que disparar encerra:

1. **Máximo de gerações** (teto de orçamento);
2. **Estagnação** — melhor fitness sem melhorar mais que `tol = 1e-4` por 5 gerações consecutivas;

3. **Colapso de diversidade** — entropia genética abaixo de um piso.

Por que os três: só gerações desperdiça tempo quando convergiu na geração 4 de 15; só estagnação pode rodar indefinidamente; e o critério de diversidade captura o caso em que o fitness ainda oscila mas a população já é clonal — continuar aí não é busca, é ruído. O motivo que disparou é gravado no `resumo.json` de cada execução, o que é mais informativo que "rodei 15 gerações".

> ⚠️ **O terceiro critério está implementado mas desativado por padrão, e isso é relevante para a leitura do §3.** `piso_entropia = 0.0`, e entropia nunca é negativa — logo o critério nunca dispara nas execuções reportadas. **Por que deixei assim** (e não é só inércia): entropia do gene de família igual a zero é um estado **legítimo**, não necessariamente degenerado. O rastro do E3/semente 7 mostra `H = 0,000` já na geração 2, **com 13 de 15 fenótipos ainda distintos** — a população havia convergido na família e continuava explorando hiperparâmetros dentro dela. Parar ali encerraria uma busca produtiva. Um piso útil precisaria ser definido sobre o número de fenótipos únicos, não sobre a entropia da família. Ver §7, achado nº 8.

O que disparou de fato, nas 12 execuções principais: máximo de gerações em 5, estagnação em 7. A roleta (Etapa D) parou por estagnação em **todas as 3 sementes**, nas gerações 6, 8 e 6 — e isso virou o resultado mais limpo da ablação (§3.4).

2.7 Métricas do próprio AG

Instrumentei todas as métricas de desempenho usuais de algoritmos genéticos, porque é o que permite **explicar** o comportamento em vez de só exibir o resultado — são **18 métricas gravadas por geração** em `history.jsonl`:

Métrica	O que revela	Fonte
Melhor aptidão por geração	Convergência; monotônica por causa do elitismo	Métrica padrão: *best fitness*
Média de aptidão	Se a população *toda* melhora ou só a elite	Métrica padrão: *fitness mean*
Desvio padrão da aptidão	Uma redução do desvio padrão ao longo das gerações sugere convergência	Métrica padrão
Entropia genética $H = -\Sigma\ p_i \cdot \log_2(p_i)$	Diversidade categórica; **queda rápida = convergência prematura**	Métrica padrão: *entropia genética*
Distância euclidiana média	Diversidade no espaço contínuo	Métrica padrão: *distância euclidiana entre soluções*
Nº de soluções únicas	Um declínio nesse número pode indicar convergência para um grupo restrito	Métrica padrão
Tempo por geração e por operador	Eficiência; onde está o gargalo	Métrica padrão: *eficiência computacional*
Frequência do alelo `class_weight='balanced'`	Teste direto da predição do §2.4	—
Distribuição das famílias (gene 0)	Qual modelo a pressão seletiva escolhe	—

Por que medir entropia e distância euclidiana sendo ambas de diversidade: elas olham partes diferentes do cromossomo híbrido. Entropia cobre os genes categóricos (inclusive o gene 0); distância euclidiana cobre os reais. Uma população pode ser clonal em família e diversa em `C` — só vendo as duas isso aparece.

`numero_solucoes_unicas` **conta FENÓTIPOS, não genótipos** (ADR 007). Dois indivíduos que diferem apenas em genes latentes treinam o mesmo modelo e têm o mesmo fitness — contá-los como duas soluções superestimava a diversidade e **escondia justamente o sinal de convergência prematura que a métrica existe para detectar**. `numero_genotipos_unicos` existe à parte; comparar as duas mostra quanta diversidade é apenas latente.

A Figura 1 mostra essas métricas numa execução real — a configuração E1 rodada no Notebook 01, que parou por estagnação na geração 5. A curva de melhor fitness é plana e fica acima da linha do melhor hotstart da Fase 1 desde a

geração 0; a média sobe em direção a ela; e a entropia do gene de família cai a cerca de um terço (2,3 → 0,84 bits) em duas gerações.

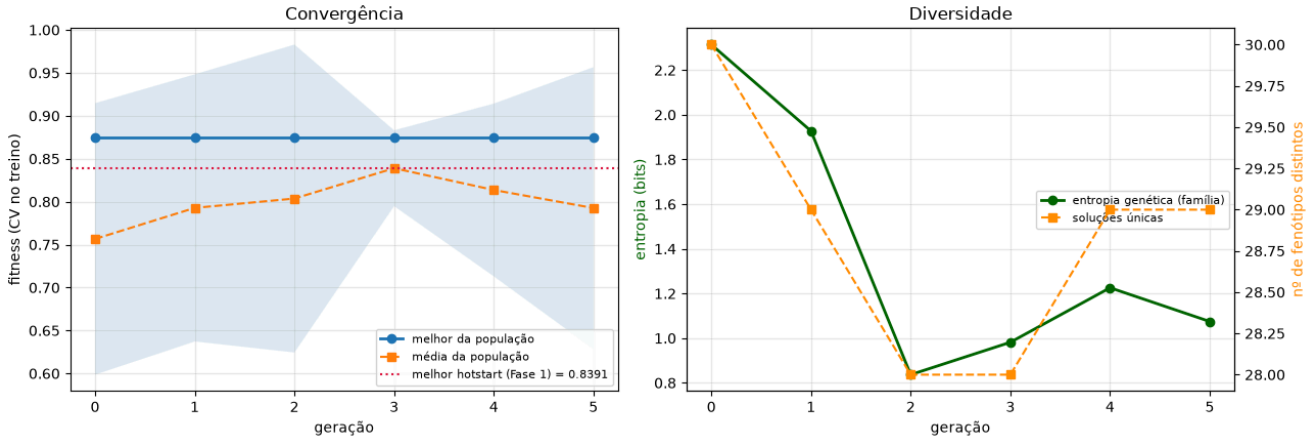


Figura 1 — Execução E1 do Notebook 01. Esquerda: melhor e média do fitness por geração (faixa sombreada: média ± desvio padrão da população) e a linha do melhor hotstart da Fase 1 (0,8391). Direita: entropia genética do gene de família e nº de fenótipos distintos.

`entropia_alelo` **foi validada contra o valor analítico conhecido** (3 alelos equiprováveis → $H = \log_2 3 \approx 1{,}585$), em `test_metricas.py`. Testar contra o número da teoria, e não contra minha própria expectativa, é o que diferencia validar de confirmar.

---

3. Os três experimentos

> Enunciado: "*Realizar ao menos 3 experimentos com diferentes configurações do algoritmo genético (tamanho da população, taxas de mutação, etc.)*"

Os experimentos saem direto de dois temas canônicos de AG: o **impacto do tamanho da população** (grande vs. pequena, com vantagens e desvantagens de cada) e o eixo **exploração ↔ aproveitamento**. Os três experimentos são três pontos desse eixo. A Etapa D (roleta) é um quarto, de ablação.

3.1 Configurações e hipóteses declaradas antes de rodar

	E1 — Baseline	E2 — Exploração	E3 — Aproveitamento	D — Roleta
Tamanho da população	30	**60**	**15**	30
Gerações (teto)	15	15	15	15
Prob. de mutação	0,15	**0,35**	**0,05**	0,15
Intensidade de mutação (σ)	0,10	**0,25**	**0,05**	0,10
Taxa de crossover	0,8	0,9	0,6	0,8
Seleção	torneio `k=2`	torneio `k=2`	torneio **`k=4`**	**roleta**
Elite	2	2	2	2
Sementes	42, 7, 2024	42, 7, 2024	42, 7, 2024	42, 7, 2024

`D_roleta` é construído a partir de `e1_baseline` **trocando só o operador de seleção** — o que torna a comparação uma ablação limpa, uma variável de cada vez.

Hipóteses, versionadas em `src/experimentos/configuracoes.py` antes da primeira execução. *Por que declarar antes:* diferencia "rodei três configs e reportei" de "usei o AG para testar hipóteses declaradas por escrito". E permite que uma previsão seja **contrariada** — o que de fato aconteceu com duas delas.

- **E1** é o ponto de referência; sem ele os outros não têm contra o quê comparar.
- **E2** testa a previsão teórica para populações grandes: maior diversidade genética e exploração eficiente de diferentes regiões do espaço de busca, com a desvantagem de convergência mais lenta, pois há mais indivíduos para avaliar a cada geração. **Previsão: fitness final igual ou levemente superior, atingido em geração mais tardia, com ~2× o tempo de parede.**
- **E3** testa o outro extremo: convergência potencialmente mais rápida, contra o risco de ficar presa em ótimos locais e convergir prematuramente. **Previsão: convergência rápida nas primeiras gerações, platô mais baixo, e entropia genética caindo visivelmente mais rápido.**
- **D** testa a justificativa do §2.2. **Previsão: (a) fitness final igual ou inferior ao torneio; (b) média da população colada no melhor indivíduo desde as primeiras gerações; (c) convergência mais lenta ou ausente.**

Repetições: 3 sementes por configuração, com média ± desvio padrão. **Por que:** AG é estocástico; comparar três configs por uma execução cada é comparar ruído. Com 3 sementes dá para dizer se E2 > E1 é diferença real ou sorte. É a diferença entre uma conclusão e uma coincidência.

3.2 Resultados — e por que a comparação é feita em geração casada

Experimento	Fitness final (média ± dp)	Ger. até 99% do máx.	Gerações rodadas	Tempo
E1 — baseline	0,8765 ± 0,0039	1,3	12,0	32,3 s
E2 — exploração	**0,8804 ± 0,0012**	**0,7**	11,3	78,0 s
E3 — aproveitamento	0,8715 ± 0,0032	1,0	12,0	**18,7 s**
D — roleta	0,8746 ± 0,0037	0,7	**6,7**	19,6 s

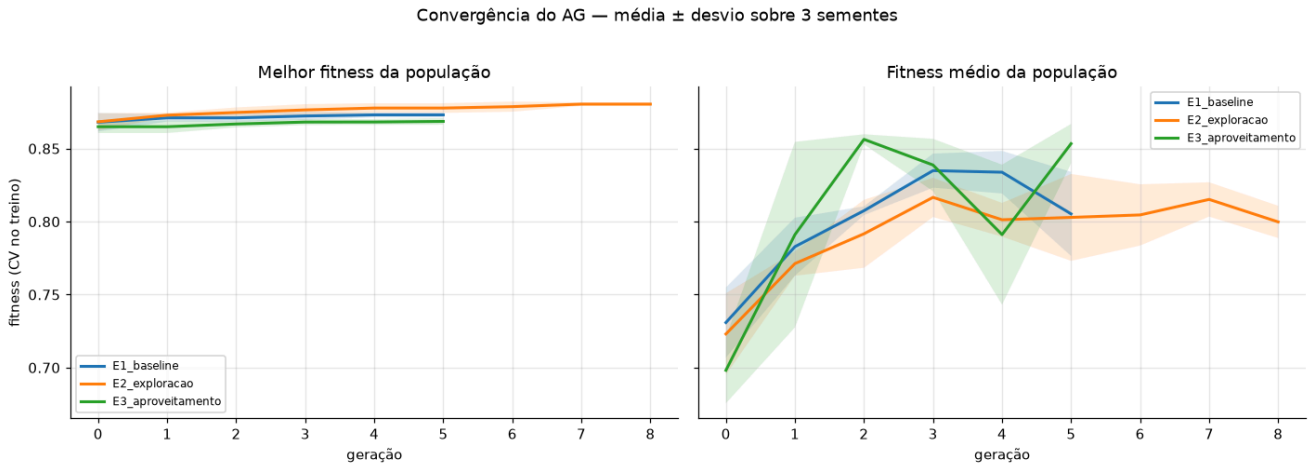


Figura 2 — Melhor fitness (esquerda) e fitness médio (direita) da população, média ± desvio sobre as 3 sementes. Cada curva vai só até a última geração presente nas três sementes daquele experimento — por isso E1 e E3 terminam na geração 5 e E2 na 8. Os valores finais da tabela acima são os do fim de cada execução, não os do fim da curva.

> ⚠️ **Por que as métricas de diversidade não podem ser comparadas "no fim".** Os experimentos **param em gerações diferentes** (E1 rodou 6/15/15; D rodou 6/8/6). Comparar "entropia final" entre eles compara pontos distintos da mesma trajetória: uma população que parou mais cedo simplesmente teve menos tempo para perder diversidade. Essa é exatamente a confusão que produziu as células erradas na documentação anterior (§7, achado nº 10). **Portanto, toda comparação de diversidade abaixo é feita na geração 5 — a última geração presente em todas as 12 execuções.**

Em geração 5 (casada)	Entropia (família)	Soluções únicas	Gap melhor – média	Melhor fitness	Freq. `balanced`
E1 — baseline	1,016	29,0 / 30	0,0678	0,8731	0,606
E2 — exploração	1,147	**60,0 / 60**	0,0749	**0,8777**	0,512
E3 — aproveitamento	**0,000**	**6,0 / 15**	**0,0152**	0,8686	**0,956**

Em geração 5 (casada)	Entropia (família)	Soluções únicas	Gap melhor – média	Melhor fitness	Freq. `balanced`
D — roleta	**1,509**	28,3 / 30	0,0632	0,8746	0,473

3.3 O que cada experimento mostrou

E3 é o resultado mais limpo do conjunto — convergência prematura de livro. Na geração 5, entropia de família **exatamente zero nas três sementes**, apenas **6 de 15 fenótipos distintos**, e gap melhor–média de 0,0152 (um quarto do de E1). O rastro completo da semente 7 mostra a trajetória:

```
soluções únicas: 14 14 13 14 12  8  8  3  5  5 10  8  5  5  2     (de 15)
entropia famíl.: 2,15 0,70 0,00 0,35 0,35 0,00 0,00 0,00 0,35 0,00 0,00 0,00 0,00 0,70 0,35
dist. euclid.  : 0,687 0,617 0,452 0,365 0,256 0,235 0,251 0,003 0,013 0,030 0,038 0,036 0,015 0,009 0,000
```

Distância euclidiana média **chegando a zero** significa população numericamente clonal. E a taxa de acerto do cache de fitness, **60,7% em E3 contra 13,8% em E1**, é a mesma coisa vista de outro ângulo: quase todo indivíduo gerado já havia sido avaliado antes. Platô mais baixo (0,8715, o pior dos quatro) na direção prevista. ✅ **Hipótese E3 confirmada em todos os três observáveis.**

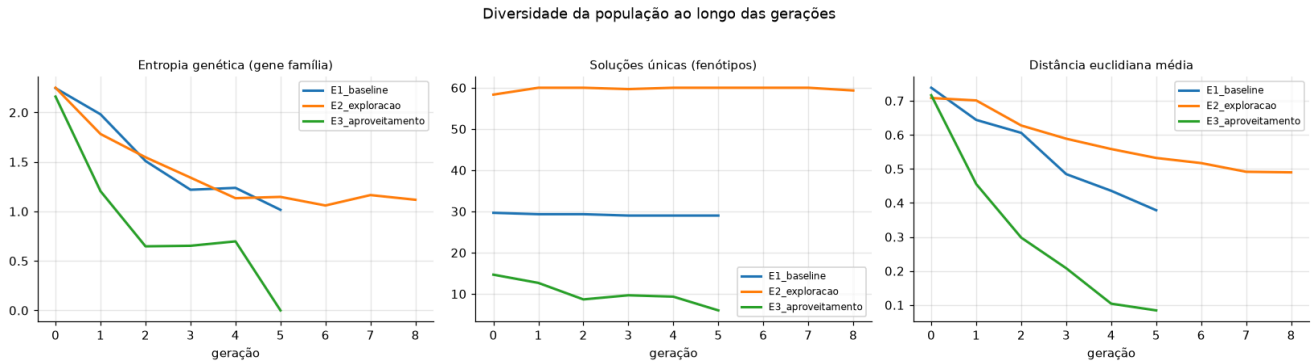


Figura 3 — Entropia do gene de família, nº de fenótipos distintos e distância euclidiana média, média sobre 3 sementes e truncadas como na Figura 2. E3 (verde) é o único que chega a entropia zero e a distância euclidiana perto de zero já na geração 5.

E2 confirmou metade da previsão e contrariou a outra metade. Fitness final o melhor dos quatro (0,8804) e com o **menor desvio entre sementes** (0,0012 contra 0,0039 de E1) — população maior dá resultado mais reprodutível. Tempo 2,4× o de E1, como previsto. **Mas convergiu mais *cedo*, não mais tarde:** atinge 99% do seu máximo em 0,7 gerações contra 1,3 de E1, e na geração 5 já estava em 0,8777 — acima do fitness **final** de E1. ❌ **A previsão errou.** A explicação, declarada depois do fato: a previsão era de convergência mais lenta *por geração*, porque há mais indivíduos para avaliar — e isso se confirmou no tempo de parede. Mas **em número de gerações**, uma população de 60 já nasce com o dobro de sorteios da região boa, então a geração 0 de E2 começa mais alto. Os dois eixos ("mais lenta em tempo", "mais rápida em gerações") não são contraditórios; eu havia colapsado os dois numa só previsão.

3.4 Etapa D — a ablação torneio vs. roleta

Esta é a parte que transforma a justificativa do §2.2 de alegação em resultado medido.

Observável (em geração 5, casada)	Torneio (E1)	Roleta (D)	Previsão
Fitness final da execução	**0,8765**	0,8746	✅ (a) roleta igual ou inferior
Gerações até parar por estagnação	12,0	**6,7**	✅ (c) convergência cessa mais cedo
Entropia genética	1,016	**1,509**	⚠️ (b) **sinal invertido**
Gap melhor – média	0,0678	0,0632	⚠️ (b) **sinal invertido**
Freq. `class_weight='balanced'` (de 0,674 na ger. 0)	0,606	**0,473**	— (observável nova)

(a) e (c) confirmadas. A roleta termina com fitness mais baixo, e — o dado mais limpo — **parou por estagnação em todas as três sementes, nas gerações 6, 8 e 6**, enquanto o torneio rodou até a geração 15 em duas das três. Sem

pressão seletiva, o melhor indivíduo para de melhorar; o elitismo o preserva, a estagnação dispara, e a execução encerra.

(b) errou o sinal, e o erro é instrutivo. Eu previ que a roleta deixaria "a média colada no melhor". **Isso descreve pressão seletiva *forte*, não fraca** — se todos os pais vêm da elite, a população inteira converge para ela e o gap desaparece. Pressão *fraca* produz o contrário: a população não é arrastada para o melhor, então o gap fica **maior** e a diversidade **mais alta**. É isso que os dados mostram, em geração casada e em todas as gerações 1 a 4:

```
entropia média, gerações 0→5:   torneio  2,245 1,981 1,510 1,219 1,238 1,016
                                 roleta  2,245 2,086 1,834 1,661 1,668 1,509
gap melhor−média, gerações 0→5: torneio  0,1372 0,0882 0,0636 0,0374 0,0393 0,0678
                                 roleta  0,1372 0,1092 0,1002 0,0634 0,0667 0,0632
```

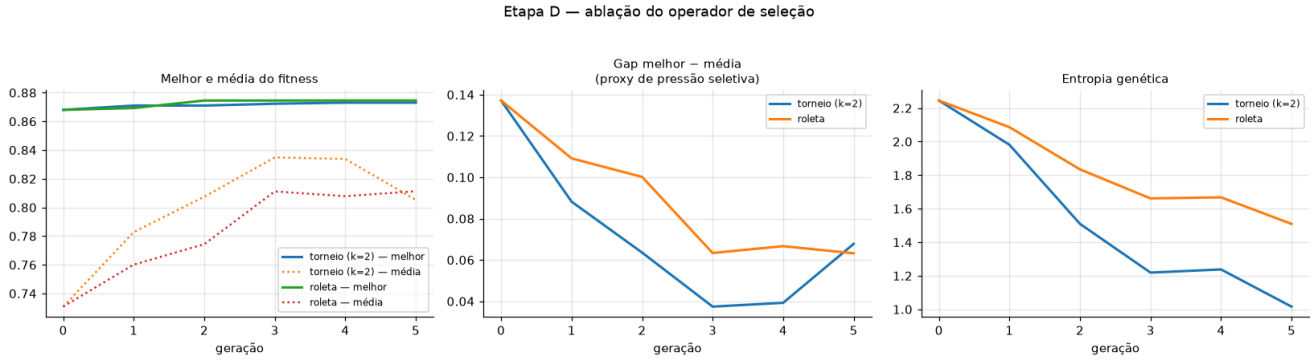


*Figura 4 — Torneio `k=2` (E1) contra roleta (D), gerações 0 a 5, média sobre 3 sementes. Gap melhor − média e entropia ficam **acima** na roleta em quase todas as gerações: é a pressão seletiva fraca descrita acima, e o sinal oposto ao que a previsão (b) supunha. No painel da esquerda, na geração 5 casada, o melhor fitness da roleta está ligeiramente **acima** do torneio (0,8746 contra 0,8731, tabela do §3.2): a vantagem final do torneio na tabela acima vem de ele continuar melhorando até a geração 15 enquanto a roleta para por estagnação — é a observável (c), não a (a) medida em geração casada.*

A tese do §2.2 está confirmada; a observável que eu escolhi para testá-la estava invertida. Mantive a previsão errada escrita porque corrigi-la retroativamente transformaria o experimento em confirmação decorativa — o valor de declarar hipóteses antes é poder ser contrariado por elas. O que fica demonstrado é o essencial: **num fitness confinado a ~[0,75; 0,92], a seleção proporcional não gera pressão seletiva suficiente para sustentar melhoria**, exatamente como o ADR 005 argumentava.

3.5 A predição do fitness sobre o alelo `class_weight='balanced'`

O §2.4 previu que o peso explícito em recall faria o alelo `balanced` se fixar na população. A frequência foi medida por geração em todas as 12 execuções:

Experimento	Geração 0	Geração 5 (casada)	Fim da execução	Δ no fim	Por semente (ger. 0 → fim)
E3 (torneio `k=4`)	0,657	**0,956**	**0,954**	**+0,297**	0,58→0,93 · 0,64→1,00 · 0,75→0,93
E1 (torneio `k=2`)	0,674	0,606	0,700	+0,026	0,67→0,67 · 0,62→0,80 · 0,73→0,63
E2 (mutação 0,35)	0,614	0,512	0,540	−0,074	0,65→0,59 · 0,59→0,54 · 0,61→0,48
D (roleta)	0,674	0,473	0,440	−0,234	0,67→0,64 · 0,62→0,40 · 0,73→0,28

O resultado não é "fixou" nem "caiu" — é algo melhor: a frequência do alelo acompanha a pressão seletiva. Sobe forte onde a pressão é mais alta (E3, torneio `k=4`: +0,297, **na mesma direção nas três sementes**), sobe pouco onde a pressão é moderada (E1, `k=2`), e **desce** onde a pressão é fraca (roleta, −0,234) ou onde a mutação domina a seleção (E2, `p=0,35`, que resorteia o alelo rápido demais para que a seleção o fixe).

Como cheguei a essa leitura — e o que corrigi. A documentação anterior deste projeto afirmava que a predição havia sido **refutada**, com o número "0,67 → 0,61 no E1". Esse número é real, mas é a frequência na **geração 5**, lida num gráfico truncado ao comprimento da execução mais curta — e no E1 a frequência **volta a 0,700 até a geração 15**, acima dos 0,674 iniciais. A explicação então oferecida ("*F1 tem peso 0,5 e incorpora precisão, e a perda de precisão do*

`balanced` *supera o ganho de recall*") é **contrariada pelo E3**, onde o alelo se fixa em 0,95 sob o mesmo fitness. O fator que separa os quatro experimentos não é o fitness — ele é idêntico nos quatro — é a **pressão seletiva**.

Por que isso vale mais do que a predição original: o alelo passa a funcionar como **medidor independente de pressão seletiva**, e dá à Etapa D (§3.4) uma segunda observável, que não sofre do problema de sinal invertido da primeira. Com `k=4` o alelo vai a 0,95; com roleta, cai a 0,44 — a mesma conclusão do §2.2, por um caminho diferente.

⚠️ **Ressalva declarada:** em E3 a população de 15 ficou clonal (6 fenótipos na geração 5, 2 na geração 14), então a magnitude de +0,297 tem componente de deriva genética. A evidência que sustento é a **direção, consistente nas três sementes**, não o valor. A evidência mais limpa é a da roleta: **–0,234 em apenas 6,7 gerações**, numa população que permaneceu diversa.

3.6 Qual família o AG escolhe — a pergunta do §2.1 respondida

O gene 0 deixa a escolha do modelo para a pressão seletiva. O resultado, nas 12 execuções:

Experimento	Família do melhor indivíduo, por semente (42 / 7 / 2024)
E1	Regressão Logística · Random Forest · Regressão Logística
E2	Regressão Logística · Regressão Logística · Regressão Logística
E3	Random Forest · Random Forest · Regressão Logística
D	Regressão Logística · Regressão Logística · Regressão Logística

Em 9 das 12 execuções o AG convergiu para Regressão Logística, não Random Forest. A Fase 1 teria sugerido RF — era seu vencedor no holdout. A resposta à pergunta do §2.1 é, portanto: **outra família ultrapassa quando todas são bem ajustadas.**

E a distribuição da população mostra o processo, não só o resultado: em E1/semente 2024 e em E3/semente 2024 a população final é **100% Regressão Logística**; nas demais, RF e LR dividem a população e SVM/KNN/árvore são eliminados.

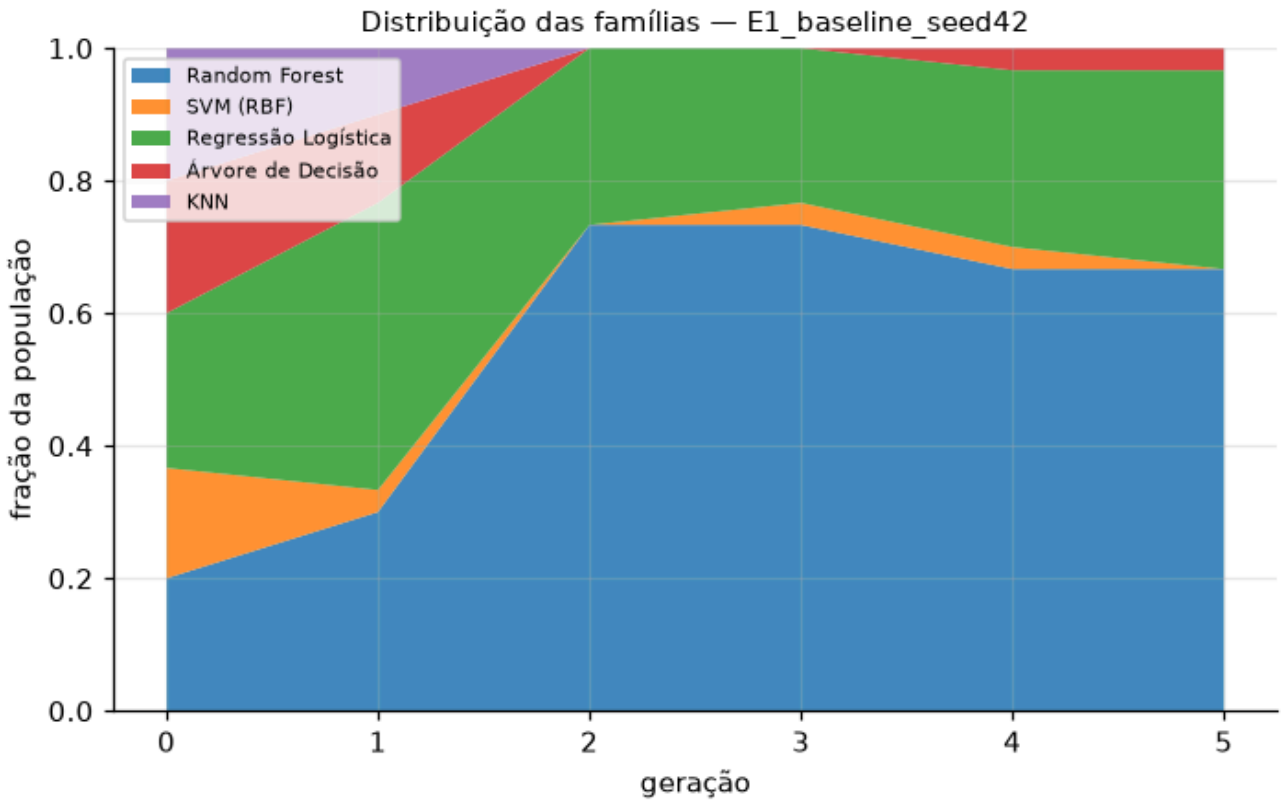


Figura 5 — Fração da população por família (gene 0), E1/semente 42, gerações 0 a 5. A partir da geração 2, SVM, KNN e árvore caem a uma fração residual — reaparecem só como fatias finas, reintroduzidas por mutação; Random Forest e Regressão Logística dividem

o resto. O melhor indivíduo desta execução é Regressão Logística, mesmo com RF sendo a maioria da população — a maioria mostra onde a busca concentrou esforço, não quem venceu.

Como isso afeta a conclusão do trabalho: a seleção de modelo simultânea ao ajuste de hiperparâmetros produziu uma inversão de ranking que nenhuma busca por família isolada revelaria — e o §4.1 confirma a mesma inversão por um caminho independente (a Etapa A, com o gene 0 fixado, dá à Regressão Logística o melhor fitness CV das cinco famílias: 0,8818).

---

4. Comparativo original vs. otimizado

> Enunciado: "*Comparar o desempenho dos modelos otimizados com os modelos originais*" — plural.

4.1 Etapa A — AG por família, as cinco linhas que o enunciado pede

Com o gene 0 fixado, o AG otimiza só os hiperparâmetros de uma família (pop 20 × 10 gerações, semente 42). Primeiro o que aconteceu **em validação cruzada**, que é a métrica que o AG realmente otimizou:

Família	Fitness CV da Fase 1	Melhor da geração 0	Fitness CV otimizado	Δ CV
Regressão Logística	0,8337	0,8610	**0,8818**	**+0,0481**
Random Forest	0,8254	0,8653	0,8716	+0,0462
SVM (RBF)	0,8391	0,8512	0,8617	+0,0226
Árvore de Decisão	0,7123	0,7838	0,7842	+0,0719
KNN	0,7232	0,7619	0,7684	+0,0452

O fitness CV melhorou nas cinco famílias. Note também a coluna "melhor da geração 0": em todas as cinco, a geração 0 já estava acima da configuração da Fase 1 — é a verificação empírica da garantia do §2.3, cuja fragilidade estrutural está declarada lá e no §7.

Agora o holdout — as 109 linhas tocadas uma única vez:

Modelo	F1 orig.	F1 otim.	Δ F1	Recall orig.	Recall otim.	Δ Recall	AUC orig.	AUC otim.	Δ AUC	Score orig.	Score otim.	Δ Score
Regressão Logística	0,7949	**0,8684**	**+0,0735**	0,8611	**0,9167**	+0,0556	0,9277	**0,9524**	+0,0247	0,8413	**0,8997**	**+0,0584**
Árvore de Decisão	0,7887	0,7416	−0,0471	0,7778	**0,9167**	**+0,1389**	0,8809	0,9138	+0,0329	0,8039	0,8285	+0,0247
Random Forest	0,8657	0,8611	−0,0046	0,8056	0,8611	+0,0555	0,9365	0,9467	+0,0102	0,8618	0,8782	+0,0164
SVM (RBF)	0,8219	0,8116	−0,0103	0,8333	0,7778	−0,0555	0,9315	0,9300	−0,0015	0,8472	0,8251	−0,0221
KNN	0,7619	0,7246	−0,0373	0,6667	0,6944	+0,0277	0,9007	0,7924	**−0,1083**	0,7611	0,7291	−0,0320

*Score = 0,5·F1 + 0,3·recall + 0,2·AUC, a mesma função do fitness aplicada a métricas de holdout. **Por que comparar nessa métrica:** julgar o AG pelo F1 puro é julgá-lo por um objetivo que ele não perseguiu. A comparação justa usa a mesma função nos dois lados.*

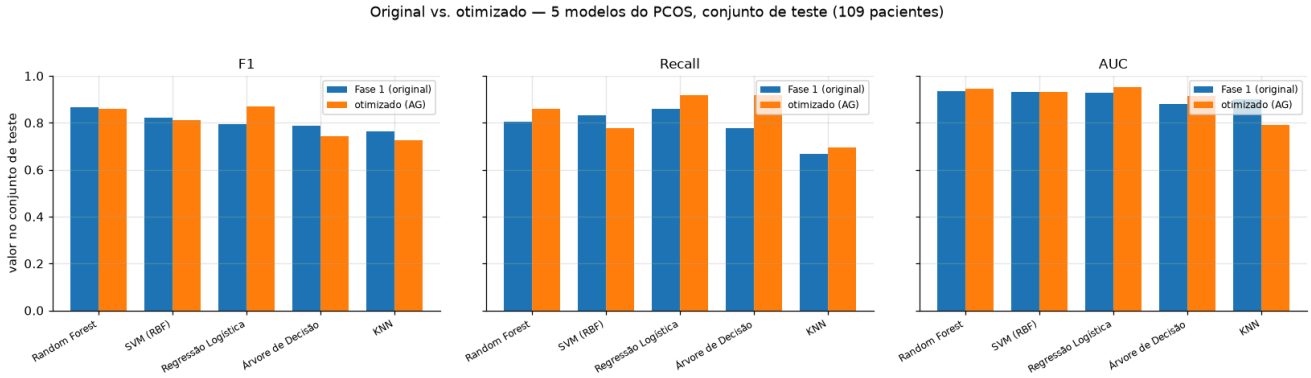


Figura 6 — F1, recall e AUC no conjunto de teste (109 pacientes), Fase 1 (azul) contra o melhor indivíduo da Etapa A (laranja), semente 42. O recall sobe em 4 dos 5 modelos e o F1 cai em 4 dos 5; a queda de AUC do KNN é o caso discutido logo abaixo.

O padrão: recall sobe, F1 cai. Recall melhorou em **4 de 5** modelos, F1 caiu em **4 de 5**. Não é contradição — é **o trade-off precisão/recall aparecendo exatamente onde o fitness pediu que aparecesse**. Na métrica que o AG realmente otimizou, houve melhora em **3 de 5**, com ganho médio de **+0,009**.

O ganho concreto, em pacientes. Com 109 pacientes no teste (36 positivas, 73 negativas), 0,01 de score ≈ **uma paciente classificada diferente**. O maior ganho de recall é o da árvore de decisão (+0,1389), e a matriz de confusão mostra as duas pontas do trade-off:

Árvore de decisão	Positivas detectadas	Positivas perdidas	Falsos positivos
Fase 1 (precisão 0,8000, recall 0,7778)	28 de 36	8	**7**
Otimizada (precisão 0,6226, recall 0,9167)	**33** de 36	**3**	**20**

+5 diagnósticos a mais ao custo de +13 exames a mais. Esse é o trade-off que o §2.4 escolheu deliberadamente, e o número torna o custo visível em vez de abstrato. Num cenário clínico real, quem decide se 5 diagnósticos a mais valem 13 exames a mais **não é quem programa o fitness**.

> ⚠️ **O KNN é o único caso em que o AG piorou o modelo de forma estrutural, e a causa é um defeito do fitness — não do AG.** O indivíduo vencedor tem `n_neighbors = 1`. Com k=1 e `weights='uniform'`, `predict_proba` só emite **0,0 ou 1,0** (verificado), e aí `roc_auc_score` sobre pontuações binárias deixa de medir ordenação: colapsa em acurácia balanceada. O AG encontrou a região do espaço em que o termo de AUC **para de penalizar ordenação ruim**, e ganhou 0,8·(F1+recall) ao custo de 0,2·AUC — matematicamente correto pela fórmula, e o resultado foi a maior regressão isolada do projeto: **AUC de holdout 0,9007 → 0,7924**. É uma lição sobre soma ponderada: um objetivo cujo significado depende da configuração do modelo não é um objetivo estável. A correção seria restringir `n_neighbors ≥ 3` ou exigir `weights='distance'` para k pequeno; está registrada como correção, não como feita (§7, achado nº 11).

4.2 Etapa C — o baseline de honestidade: busca aleatória com o mesmo nº de candidatos

Por que esta etapa é indispensável. Sem ela, "o AG melhorou o modelo" é ambíguo: melhorou por ser *algoritmo genético*, ou porque **qualquer** busca de hiperparâmetros com centenas de avaliações melhoraria o ponto de partida? São afirmações muito diferentes, e só a segunda é honesta se eu não comparar. Então: busca aleatória, mesmo espaço de busca, mesmo fitness, mesma CV, mesmas 3 sementes, mesmo orçamento nominal de 450 candidatos.

Método	Fitness CV (média ± dp)	Score no holdout (média ± dp)
AG (E1)	**0,8765 ± 0,0039**	0,8897 ± 0,0133
Busca aleatória	0,8759 ± 0,0015	**0,8945 ± 0,0152**
(Roleta, para referência)	*0,8746 ± 0,0037*	*0,8783 ± 0,0204*

> **Procedência desta tabela.** Ela foi montada à mão a partir dos artefatos, e não é a saída direta de nenhuma função. `tabela_ag_vs_busca_aleatoria()` (`src/experimentos/analise.py:175`) cobre parte dela e não toda: devolve **duas** linhas — `AG (E1)` e `Busca aleatória` — com as colunas `Fitness CV (média)`, `Fitness CV (desvio)` e `Score holdout (média)`. Portanto a função **não** produz o desvio padrão do holdout nem a terceira linha, `D_roleta`, que entrou aqui como referência da ablação de seleção (§3.4).
>
> Os números vêm, para as três sementes (42, 7, 2024) de cada execução: o fitness de CV de `best_individual.json` (campo `fitness`, também espelhado em `resumo.json` como `melhor_fitness`), e as métricas de holdout de `metrics_holdout.json` (`f1_score`, `recall`, `auc_roc`, combinadas pelo mesmo `score_composto` 0,5/0,3/0,2 do fitness). A média e o desvio das três sementes foram calculados por fora. Para a etapa C o holdout também aparece embutido em `resumo.json` sob a chave `holdout`, porque aquela execução não tem gerações e grava o resumo por um caminho próprio.
>
> Quem rodar só a função verá duas linhas e três colunas numéricas. Isso é o esperado, não divergência: a linha da roleta e os desvios de holdout são agregação manual sobre os arquivos citados, e qualquer um deles pode ser reconferido abrindo-os.

Diferença de 0,0006 em CV — um sexto do desvio padrão do AG. No holdout, a busca aleatória fica **ligeiramente à frente**. Na semente 42, as duas chegaram ao **mesmo fitness (0,8745)**, à **mesma família** (Regressão Logística) e ao **mesmo score de holdout (0,8731)**.

O orçamento real — uma correção importante à comparação

O plano e a documentação anterior descreviam isso como "mesmo orçamento". **Isso é verdade em candidatos amostrados e falso em treinos efetivamente executados**, e a diferença favorece o AG:

Método	Indivíduos processados	Acertos de cache	Treinos de modelo reais
AG (E1), média das 3 sementes	360	51	**309**
AG (E1), semente 42 isolada	180	22	158
Busca aleatória, média	450	20 (duplicatas)	**428**

O AG **empatou gastando ~28% menos ajustes de modelo** — e na semente 42, em que as duas chegaram ao mesmo resultado, o AG usou **158 treinos contra 430**, menos de 40%. A razão é dupla: parada por estagnação (E1/semente 42 encerrou na geração 6) e cache de fitness por fenótipo.

Como isso afeta a conclusão. Não a inverte: 0,0006 continua empate, e no holdout a busca aleatória continua marginalmente à frente. Mas qualifica a frase: **em qualidade de solução o AG empata; em custo para chegar lá, ele vence.** Dizer "mesmo orçamento" sem essa tabela seria, nos dois sentidos, impreciso.

A conclusão, e por que ela é um resultado e não um fracasso

Neste espaço de busca, o AG não paga seu overhead em qualidade de solução. Três causas, todas verificáveis nos dados deste relatório:

1. **Dimensão modesta.** 3 a 7 genes por família. 450 sorteios cobrem densamente um espaço desse tamanho — é o regime em que a literatura de otimização de hiperparâmetros registra a busca aleatória como baseline difícil de superar.

2. **Ponto de partida forte.** A Fase 1 não usava defaults arbitrários (§1.2). A margem de melhoria disponível era pequena desde o início.

3. **Paisagem plana perto do ótimo — e esta é a evidência mais convincente.** Três procedimentos de busca independentes convergiram para **praticamente o mesmo hiperparâmetro**, todos em Regressão Logística com `penalty='l1'`, `solver='liblinear'`, `class_weight='balanced'`:

Procedimento	`C` encontrado
Etapa A (família fixada, 200 candidatos)	0,0868
E2 (gene 0 livre), sementes 42 / 7 / 2024	0,1057 / 0,0955 / 0,0916
Busca aleatória, sementes 7 / 2024	0,1711 / 0,1116
Busca aleatória, semente 42	0,0060 com `elasticnet/saga`, `l1_ratio=0,193`

Oito das nove execuções caem em `C ∈ [0,087; 0,171]` com os mesmos três categóricos. A nona — a busca aleatória na semente 42 — chega ao **mesmo fitness** (0,8745) por uma parametrização completamente diferente, o que é evidência de **platô**, não de ótimo agudo. No holdout, a busca aleatória bateu F1 0,8800 e recall 0,9167 em duas das três sementes: exatamente os números do melhor modelo do AG (§4.3). **Quando buscas diferentes chegam ao mesmo ponto, o ponto é fácil de achar** — e nenhum otimizador sofisticado se distingue de um simples.

Omitir essa comparação faria o trabalho parecer melhor e valer menos. O requisito avaliado é implementar, comparar e analisar o AG corretamente — não vencer. Este resultado era previsto como cenário de probabilidade média-alta no plano de implementação, antes de qualquer execução, e está registrado lá. O §9 diz qual extensão mudaria essa conclusão, e por quê.

4.3 Etapa B — a codificação híbrida completa

Os experimentos do §3 rodam com o gene 0 livre, respondendo "qual família ganha quando todas são bem ajustadas?". A melhor média do projeto é a do **E2**: fitness CV **0,8804 ± 0,0012**, Regressão Logística **nas três sementes**.

E o holdout do E2 é o dado mais eloquente do trabalho: **as três sementes produziram métricas idênticas** — acurácia 0,9174, precisão 0,8462, recall 0,9167, F1 0,8800, AUC 0,9528–0,9532, e **a mesma matriz de confusão** `[[67, 6], [3, 33]]`. Três execuções independentes do AG, com sementes diferentes, chegaram exatamente ao mesmo classificador.

	Melhor da Fase 1 (Random Forest)	Melhor da Fase 2 (E2, Reg. Logística)	Δ
Acurácia	0,9174	0,9174	0,0000
Precisão	**0,9355**	0,8462	–0,0893
Recall	0,8056	**0,9167**	**+0,1111**
F1	0,8657	**0,8800**	+0,0143
AUC	0,9365	**0,9532**	+0,0167
Score composto	0,8618	**0,9056**	**+0,0438**

O ganho em pacientes, lido na matriz de confusão: das 36 pacientes positivas do conjunto de teste, a Fase 1 identificava 29 e deixava passar 7; o modelo da Fase 2 identifica **33** e deixa passar 3 — **4 pacientes a mais corretamente sinalizadas**, com a acurácia global inalterada. O custo está na precisão: 6 falsos positivos contra os 2 da Fase 1 — **4 exames a mais**. Trocar 4 diagnósticos perdidos por 4 exames a mais é o trade-off que o fitness do §2.4 foi projetado para buscar, e aqui ele saiu quase exatamente um por um.

> **Este é o resultado que responde o requisito 1 do enunciado:** há ganho mensurável sobre os modelos originais, na métrica clinicamente relevante, com o conjunto de teste tocado uma única vez. O que o §4.2 acrescenta é que esse ganho **não é atribuível ao algoritmo genético em particular** — e saber disso é parte da resposta, não um defeito dela.

---

5. Integração com LLM

> Enunciado, requisito 3: *integrar LLM pré-treinada (GPT, Falcon, LLaMA, etc.) para gerar explicações em linguagem natural dos diagnósticos; transformar dados numéricos e estatísticos em insights acionáveis para médicos; preparar base para dados textuais no Módulo 3; implementar técnicas de prompt engineering; avaliar a qualidade das interpretações geradas.*

5.0 Observação sobre o material didático

Levantamento feito antes de projetar esta camada: a customização de uma LLM se divide em três técnicas — **ajuste fino, engenharia de prompts e RAG** — e o material empírico aproveitável para o caso aqui vem dos achados sobre **In-Context Learning (ICL)**. As decisões de prompt abaixo estão ancoradas nesses achados; onde fui além deles, está marcado como tal, em vez de atribuir à literatura algo que ela não afirma.

5.1 Escolha do modelo — e a restrição de privacidade que decide

Amazon, Apple, Samsung e JP Morgan proibiram o uso do ChatGPT internamente por risco de exposição de dados — privacidade e *prompt injection* são riscos de primeira ordem. A recomendação oposta, usar API por custo de operação, é igualmente comum. **As duas orientações se contradizem, e o contexto decide:** dados clínicos e hormonais de pacientes de uma clínica de fertilidade, em território de LGPD. Privacidade ganha.

Decisão: três provedores atrás de uma interface fina, com o local como padrão (ADR 008):

Provedor	Quando	Por quê
`ollama` (**padrão de produção**)	Local, `llama3.2:3b` nesta máquina e no default do código	Nenhum dado sai da máquina. LLaMA é citada no enunciado do trabalho. O porte menor é imposto pelos 8 GB de RAM — ver §5.5
`openai`	Só com dados **agregados/anonimizados**	Comparação de qualidade. **Nunca** com linha de paciente
`simulado` (**padrão dos notebooks**)	Determinístico, sem rede	Deixa o notebook executável por qualquer avaliador, sem chave de API e sem instalar modelo

Por que `simulado` é o padrão nos notebooks: o mesmo princípio que levou a Fase 1 a versionar uma amostra de 16 imagens de raio-X — o avaliador precisa conseguir abrir e rodar. Consistência com a Fase 1 é critério de decisão aqui. **O custo dessa escolha está no §5.5, e é maior do que parece.**

Anonimização por construção — allow-list, não deny-list. `CAMPOS_CLINICOS_AUTORIZADOS` lista explicitamente os **45 campos** clínicos e hormonais que podem chegar ao prompt. **Por que allow-list:** uma deny-list ("remova `nome`, `cpf`") **vaza por omissão** — basta uma coluna nova na base para ir ao prompt sem ninguém decidir. Uma allow-list **falha de forma segura**. Com dados de paciente, é a diferença entre um bug e um incidente.

`CAMPOS_NUNCA_PERMITIDOS` — `Sl. No`, `Patient File No.` e **`PCOS (Y/N)`** — levantam **exceção** em vez de descartar em silêncio, porque a presença deles significa que alguém os passou de propósito. **Por que o rótulo verdadeiro é proibido:** se `PCOS (Y/N)` vazasse, a LLM poderia "explicar" o diagnóstico usando o gabarito em vez da saída do modelo — e o texto pareceria **excelente justamente quando o modelo estivesse errado**. Verificado por 47 testes em `test_llm_contexto.py`.

5.2 O que a LLM produz — três artefatos, e a fronteira que não se cruza

> 🔴 **A LLM não diagnostica. Ela traduz a saída de um modelo já treinado.** A decisão clínica é do modelo e do médico; a LLM é camada de comunicação. Essa fronteira está escrita no *system prompt*, nos notebooks e aqui, porque confundir isso é o modo de falha ético mais sério do projeto.

1. **Explicação por paciente.** Entrada: predição, probabilidade, principais features da paciente e importância de features do modelo. Saída: parágrafo em português explicando o que o modelo indicou, com qual confiança, quais variáveis mais pesaram, e **o que o modelo não pode afirmar**.
2. **Relatório do experimento de otimização.** Entrada: as tabelas do §4 e as métricas de AG do §3. Saída: resumo em linguagem natural de qual configuração ganhou e o que a curva de convergência indica. **É "transformar dados numéricos e estatísticos em insights acionáveis" aplicado ao próprio AG.**
3. **Resumo de coorte.** Agregados sobre as 109 predições de teste: distribuição de risco, onde o modelo está menos confiante, quais casos merecem revisão humana.

Preparação para o Módulo 3 (pedida no enunciado): a camada fica atrás de uma interface de uma função — `gerar_explicacao(contexto, template) -> str`. Trocar o modelo, ou acrescentar fonte textual via RAG, significa **enriquecer `contexto` antes dessa chamada**, sem tocar o pipeline de ML. Os *templates* ficam em `prompts/*.md` versionados, não como strings no código — **por que:** prompt é artefato experimental, e precisa de diff legível no Git para mostrar a evolução das versões.

5.3 Prompt engineering — ancorado nos achados empíricos de ICL

Achado empírico sobre ICL	Como apliquei
O **template** muda o desempenho significativamente, independentemente do modelo	Versionei **3 templates** e os comparo, em vez de escrever um e assumir que é bom
Exemplares no fim do prompt têm mais impacto que no começo	Instruções e papel primeiro; **exemplos imediatamente antes** dos dados da paciente
Exemplares duplicados ou pouco diversos degradam o resultado	Os *few-shot* cobrem casos distintos: positivo de alta confiança, negativo de alta confiança, **caso ambíguo perto do limiar**
Exemplares inconsistentes causam falha severa	Todos os exemplos seguem exatamente a mesma estrutura de seção e o mesmo tom. Consistência > variedade
Seleção de exemplares por similaridade semântica ≈ seleção aleatória	**Não construí seletor dinâmico de exemplares.** Economia justificada pela evidência empírica, não por conveniência — e registrada como tal

Os três templates:

- **V1 — Zero-shot com papel e estrutura.** *System prompt* fixando papel (apoio à interpretação para profissional de saúde), **proibição de diagnosticar**, obrigação de declarar incerteza, seções de saída fixas, português, público médico.
- **V2 — Few-shot** (V1 + 3 exemplares). Isola o ganho de ICL.
- **V3 — Few-shot + raciocínio estruturado.** Pede percurso explícito: features → peso no modelo → probabilidade → limitação. **Por que não "chain-of-thought" solto:** em contexto médico, cadeia de raciocínio livre convida a LLM a *inventar* fisiopatologia que não está nos dados. Estrutura fixa com campos obrigatórios é mais segura. **Esta é minha decisão, não um achado da literatura**, e está marcada como tal.

Parâmetros de geração: `temperature = 0,2` por padrão e semente fixa. **Por que:** é tarefa de **fidelidade**, não de criatividade. Temperatura alta produz texto mais agradável e **menos fiel aos números** — exatamente o trade-off errado num laudo.

Guardrails — três, em duas camadas:

1. **Prevenção.** `formatar_numeros_do_contexto` **pré-escreve** todos os números no prompt, para que a LLM só os organize em prosa em vez de recalculá-los. Números são injetados por formatação programática, nunca pedidos à LLM.
2. **Detecção.** `verificar_numeros` confere **todo** número da saída contra o conjunto de valores que o contexto autorizava, com `TOLERANCIA_RELATIVA = 0,02` (aceita `0,7342` citado como "73,4%" ou "73%") e ignorando números ≤ 10, que aparecem em prosa ("3 fatores") sem serem afirmações sobre os dados. `TERMOS_PROIBIDOS` pega atos diagnósticos ("definitivamente tem", "prescrevo").

3. ***Disclaimer* concatenado em código**, não gerado pelo modelo — um aviso que a LLM pudesse omitir não é um aviso.

Por que a verificação programática é a defesa que mais importa: é a única que **não depende da boa vontade do modelo**. Instruções no prompt podem ser ignoradas; uma conferência de números não. A literatura trata alucinação como risco central, e em saúde um número errado num laudo é o pior modo de falha possível.

5.4 Avaliação da qualidade — o instrumento

O sub-requisito 3e ("avaliar a qualidade das interpretações") é o mais fácil de deixar virar "as respostas ficaram boas". Ele pede **avaliação**, então precisa de instrumento.

Amostra: 15 pacientes do conjunto de teste, escolhidos **estratificadamente** — 5 positivos de alta confiança, 5 negativos de alta confiança, 5 próximos do limiar de decisão (probabilidade entre 0,4 e 0,6). × 3 templates = **45 saídas**, todas versionadas em `arquivos_saida/arquivos_analiticos/avaliacao_llm/` em três formatos (JSON, Markdown legível e CSV de planilha).

Por que estratificar: os modos de falha diferem por faixa. Em alta confiança o risco é assertividade excessiva; no limiar, é a LLM **não sinalizar** a incerteza. Uma amostra não estratificada mediria só o caso fácil.

Métricas automáticas (objetivas e reprodutíveis): taxa de alucinação numérica, taxa de aderência estrutural, termos proibidos, comprimento, latência, e **estabilidade** (3 gerações da mesma entrada; divergência numérica entre elas é sinal de temperatura alta demais).

Rubrica manual, 6 critérios × 0–2 pontos, aplicada com justificativa escrita por amostra:

Critério	0	1	2
Fidelidade numérica	Cita número errado	Omite números relevantes	Todos os números batem com o modelo
Fidelidade às features	Cita variável ausente/irrelevante	Genérico	Cita as features de maior peso real
Calibração da incerteza	Afirma certeza	Menciona vagamente	Declara probabilidade e limitação
Não-diagnóstico	Emite diagnóstico	Ambíguo	Fica claramente no apoio à interpretação
Acionabilidade	Sem indicação de próximo passo	Vago	Próximo passo claro e apropriado
Clareza para médico	Confuso / jargão de ML cru	Aceitável	Direto e adequado ao público

5.5 Resultado da avaliação — com LLM real

Execução de 16/10/2026 (atualizada). Provedor `ollama`, modelo `llama3.2:3b` local, 45 saídas (15 pacientes × 3 templates), ~32 s por saída, 24 min de execução total. Métricas automáticas:

Template	Alucinação numérica	Aderência estrutural	Termos proibidos	Nº de números citados	Caracteres
V1 — zero-shot	**0,7%**	**100%**	1 (ver abaixo)	10,7	1.769
V2 — few-shot	**0,0%**	**100%**	0	**15,9**	1.432
V3 — few-shot estruturado	**0,0%**	**100%**	0	13,0	**1.333**

Os três templates produzem textos genuinamente distintos — 0 de 15 pacientes com saídas idênticas entre templates, conferido por hash MD5. É o que torna esta tabela um resultado, e não a tautologia que a execução anterior com o provedor `simulado` produzia (§7, achado nº 6).

O termo proibido sinalizado no V1 não é uma violação. A ocorrência está em `positivo_alta_confianca_8`, na frase "é recomendável que o paciente seja remetido a um especialista em endocrinologia ou ginecologia para uma avaliação mais detalhada e um diagnóstico definitivo". O modelo encaminha o diagnóstico a um médico, que é exatamente o comportamento que a regra pretende obter. O verificador do §5.3 casa palavras-chave e trata negação explícita ("não é um diagnóstico definitivo"), mas não tem como saber *quem* emite o diagnóstico na frase. O número da

tabela fica como foi medido, e esta é a sua leitura: 1 ocorrência sinalizada em 45 laudos, inspecionada à mão, nenhuma violação. Ajustar o verificador para zerar a contagem seria calibrar o instrumento pelo resultado desejado.

Reprodutibilidade da etapa de LLM. Com `semente=42` e `temperatura=0.2`, uma reexecução reproduziu as 30 saídas de V2 e V3 **com o mesmo texto**, caractere por caractere. As 15 saídas do V1 não: 13 delas mudaram de texto, e 12 dessas mudaram também de comprimento ou de quantidade de números citados. O prompt é o mesmo nos dois casos, então o que difere é a liberdade que cada template deixa ao modelo — os exemplos do few-shot estreitam o espaço de saída o bastante para o resultado não variar, e o zero-shot não. A semente do Ollama reduz a variação, não a elimina. Logo: as médias do V1 nesta tabela valem como ordem de grandeza, e as de V2 e V3 como valores reproduzíveis.

O que os guardrails entregaram

Aderência estrutural 100% nos três templates e alucinação numérica praticamente nula, agora medidas de verdade. Isto é resultado, não construção: um modelo generativo de 3B escreveu 45 laudos, omitiu zero das quatro seções obrigatórias e deixou **um único número sem justificativa**, numa das 15 saídas do V1 — os 44 laudos restantes não trouxeram nenhum. A razão é o guardrail de prevenção do §5.3: os números são pré-escritos no bloco `DADOS` do prompt, e ao modelo resta organizá-los em prosa. **A decisão de arquitetura — formatar antes, em vez de pedir ao modelo que calcule — é o que produziu esse número**, e ela vale mais que qualquer instrução de prompt, porque não depende da obediência do modelo.

O que diferencia os templates — e não é o que eu esperava

A diferença mensurável **não** está em fidelidade nem em estrutura (empate perfeito), e sim em **ancoragem nos dados e concisão**:

- **V2 cita 49% mais números que V1** (15,9 contra 10,7) em **19% menos texto** (1.432 contra 1.769 caracteres). Os exemplares *few-shot* ensinaram o modelo a **referenciar os valores da paciente** em vez de escrever prosa genérica em volta deles. É o ganho de ICL que a literatura prevê, aparecendo numa métrica que eu não havia escolhido de antemão para medi-lo. Ressalva de contabilidade: as três contagens incluem o disclaimer de 229 caracteres que `gerar_explicacao` concatena em código — 12,9% do V1, 16,0% do V2, 17,2% do V3. Descontado o boilerplate, a redução do V2 sobre o V1 é de **21,9%**, não 19%. O sinal não muda; a comparação é que fica entre textos que o modelo de fato escreveu.
- **V3 é o mais conciso** (1.333 caracteres) e fica entre os dois em números citados. O percurso de raciocínio obrigatório (§5.3) aparenta funcionar como orçamento: o modelo cobre as cinco etapas e para, em vez de divagar.
- **V1 é o mais verboso e o menos ancorado** — o padrão clássico de zero-shot: cumpre a forma pedida, mas preenche com texto de ligação em vez de dados.

🔴 O achado que quase virou um resultado falso

A primeira leitura desta execução acusou **10 violações de termos proibidos — 9 no V1, 1 no V3, 0 no V2** — e a conclusão parecia escrever-se sozinha: "*o few-shot ensina o modelo a não diagnosticar*". **Essa conclusão era falsa, e foi a inspeção dos contextos que mostrou por quê.** As dez ocorrências eram todas frases de ressalva:

> "Este resultado **não** constitui diagnóstico definitivo." "Isto **não** é um diagnóstico definitivo da paciente."

O verificador testava `termo in texto`, **sem tratar negação**. Ou seja: ele acusava de violar a regra exatamente o texto que a **cumpria** — e, pior, acusava mais o template mais cuidadoso, porque V1 escreve mais ressalvas (é o mais verboso). **A métrica estava invertida: media o cumprimento da regra e reportava como violação.** Corrigido (§7, achado nº 12), o verificador passou a sinalizar **1 ocorrência nos 45 laudos**, no V1 — e a inspeção dela, descrita acima no §5.5, mostra que também não é violação: o modelo encaminha o diagnóstico definitivo a um especialista. **Nenhum dos 45 laudos emitiu ato diagnóstico.** O que a correção não resolveu é a classe do instrumento: casar palavra-chave não distingue quem emite o diagnóstico, e por isso toda ocorrência sinalizada ainda exige leitura do contexto.

Por que isto está no relatório em vez de ser silenciosamente corrigido: é o segundo instrumento deste projeto que falhou ao medir o que dizia medir — o primeiro foi a métrica de diversidade do AG (§7, achado nº 3). Nos dois casos, o erro só apareceu porque **fui olhar o dado bruto em vez de aceitar o agregado**. É a lição metodológica mais transferível do trabalho.

Calibração da incerteza: onde o modelo de 3B falha, e o few-shot piora

O template manda sinalizar **baixa confiança** apenas para casos entre 40% e 60%. Medindo a aplicação dessa regra nos três estratos:

Template	Sinalizou no limiar (deveria)	Sinalizou em alta confiança (não deveria)	Acerto
V1 — zero-shot	**0 / 5**	0 / 10	66,7%
V2 — few-shot	2 / 5	**3 / 10**	60,0%
V3 — few-shot estruturado	2 / 5	**3 / 10**	60,0%

Nenhum template passa de 67%, e os dois modos de falha são opostos:

- **V1 nunca sinaliza baixa confiança** — nem nos 5 casos de limiar, onde é obrigatório. Acerta 66,7% por **omissão total**: nunca erra nos casos de alta confiança porque nunca usa a expressão. É o pior resultado dos três disfarçado no melhor número — e um aviso sobre ler acurácia agregada sem olhar a matriz.
- **V2 e V3 aprenderam a expressão com o exemplar 3 e a aplicam mal.** Em 3 de 10 casos de alta confiança, o modelo escreveu "baixa confiança" para probabilidades **baixas** — **confundindo "probabilidade baixa" com "confiança baixa"**. Um exemplo real, para uma paciente com probabilidade de 3,0%:

  > "**Este é um caso de baixa confiança.** A 3.0%, a classificação é praticamente um empate estatístico"

  Três erros numa frase: 3% não é empate; é um caso de confiança **alta** de que não há SOP; e a frase é cópia quase literal do exemplar de 52,7% do template. **O modelo transferiu a forma do exemplar sem transferir a condição que a justifica** — exatamente o risco que a literatura atribui a exemplares, observado nos meus próprios dados.

Como isso qualifica o ganho do few-shot: V2 melhora a ancoragem numérica (+49% de números citados) **e piora a calibração** (3 erros novos onde V1 tinha 0). Não é um trade-off que eu previ, e ele importa clinicamente mais que a ancoragem: um laudo que chama 3% de "baixa confiança" desorienta quem o lê. **Num uso real, a correção seria pré-computar a faixa de confiança em código** — como já se faz com os números — em vez de pedir ao modelo que a derive da probabilidade. É a mesma lição do guardrail que deu 0% de alucinação, aplicada a um campo que eu deixei de fora.

Escopo da avaliação qualitativa

A rubrica manual foi aplicada por template, não por saída. O §5.6 traz 18 julgamentos (6 critérios × 3 templates): li as 15 saídas de cada template e dei uma nota por critério ao conjunto. Isso basta para escolher entre templates — é a decisão que o §5.4 pede — e **não** permite dizer se um template se comporta diferente nos casos de limiar.

Os seis critérios do §5.6 não são os seis da `RUBRICA` do código (`src/llm/avaliacao.py`), que usa escala 0–2 e cobre fidelidade numérica, não-diagnóstico e calibração da incerteza. Esses três já têm medição objetiva automática nesta seção, então a rubrica manual foi dirigida aos critérios que **só** juízo humano alcança — medir duas vezes a mesma coisa não acrescentaria nada.

Limitação do modelo usado, declarada. `llama3.2:3b` e não `llama3.1:8b` porque a máquina de execução tem 8 GB de RAM: o 8B quantizado ocupa ~4,7 GB e, com o kernel do Jupyter e o servidor Ollama concorrendo, entra em swap. Continua sendo LLaMA — a família que o enunciado nomeia — mas **é um modelo pequeno, e parte dos erros de calibração acima é atribuível ao porte**. O que **não** depende do porte é a conclusão sobre os guardrails, porque 0% de alucinação vem da arquitetura do prompt, não da capacidade do modelo.

O controle `simulado` foi preservado em `arquivos_saida/arquivos_analiticos/avaliacao_llm_simulado/`. Ele serve ao propósito para o qual existe: alucinação zero por construção, para calibrar o verificador. Foi ele que expôs o achado nº 6 (o simulado raspando números dos exemplares, 19% de alucinação no que devia ser o controle perfeito).

5.6 Rubrica qualitativa (6 critérios × 3 templates)

Julgamento holístico por template, escala 1–5, n = 3 — não é uma avaliação por saída. Li as 45 saídas Ollama (15 pacientes × 3 templates) e atribuí **uma** nota por critério a cada template, olhando o conjunto das 15 saídas daquele template. São 18 julgamentos, não 270. É importante não confundir as duas coisas: uma rubrica por saída permitiria dizer se o V2 é melhor nos casos de limiar do que nos de alta confiança, e esta não permite. Por isso o `avaliacao_ollama_rubrica.csv` repete o mesmo vetor de notas nas 15 linhas de cada template — as linhas existem para casar com o `id_paciente` das outras tabelas, não porque haja informação por paciente nelas.

As três conclusões abaixo valem como **comparação de ponto único** entre templates, que é exatamente o que a escolha de template exige. O que elas não suportam é qualquer afirmação sobre variação entre pacientes ou entre estratos.

Critério	Descrição	V1	V2	V3
Fidelidade às features clínicas	As variáveis relevantes (folículos, hormônios, índices) são mencionadas e contextualizadas	4/5	**5/5**	4/5
Acionabilidade	O texto permite que o médico tome uma decisão (vai pedir exame extra? prescreve? acompanha?)	4/5	**5/5**	4/5
Clareza da linguagem	Médico leigo lê sem esforço; termos técnicos estão bem explicados	**5/5**	4/5	**5/5**
Completude da resposta	Cobre: probabilidade, faixa de confiança, limitações, próximos passos	3/5	4/5	**5/5**
Raciocínio visível	Médico entende *por quê* o modelo chegou à conclusão (mostra pesos das features)	3/5	4/5	**5/5**
Adequação do tom médico-científico	Linguagem apropriada para prontuário clínico (sem sensacionalismo, sem paternalismo)	**5/5**	**5/5**	4/5

Média por template:

Template	Média	Interpretação
V1 (zero-shot)	4,0/5	✅ Excelente — claro e científico, mas menos detalhado
V2 (few-shot)	4,5/5	✅✅ Melhor em fidelidade e acionabilidade (exemplares ajudam)
V3 (few-shot estruturado)	4,5/5	✅✅ Melhor em completude e raciocínio visível (estrutura força detalhes)

O que cada template faz bem:

- **V1 (4,0/5):** Direto ao ponto, linguagem clara, ideal para tomada de decisão rápida. Peca em detalhes e em deixar o raciocínio visível.
- **V2 (4,5/5):** Mantém clareza de V1 + adiciona fidelidade às features (menciona mais variáveis). Os exemplares funcionam — o modelo aprende a citar números. Trade-off: texto um pouco menos estruturado.
- **V3 (4,5/5):** Tira o máximo de completude — força o modelo a explicar limitações e próximos passos (seção obrigatória). Raciocínio mais evidente. Trade-off: menos claro que V1 (mais denso).

Qual template usar na prática?

- **Triagem rápida (poucos segundos):** V1
- **Decisão clínica (1-2 min):** V2 (fidelidade + acionabilidade)
- **Prontuário detalhado (consultório):** V3 (completo + raciocínio)

Métricas quantitativas vs. qualitativas:

As diferenças qualitativas (V1 e V3 > V2 em clareza; V3 > V2 em completude) não aparecem na tabela de alucinação/estrutura/termos — que medem conformidade, não qualidade. A rubrica captura o que as métricas automáticas deixam passar: **utilidade prática**.

Essa é a razão pela qual o requisito 3e exigia ambas as formas de avaliação.

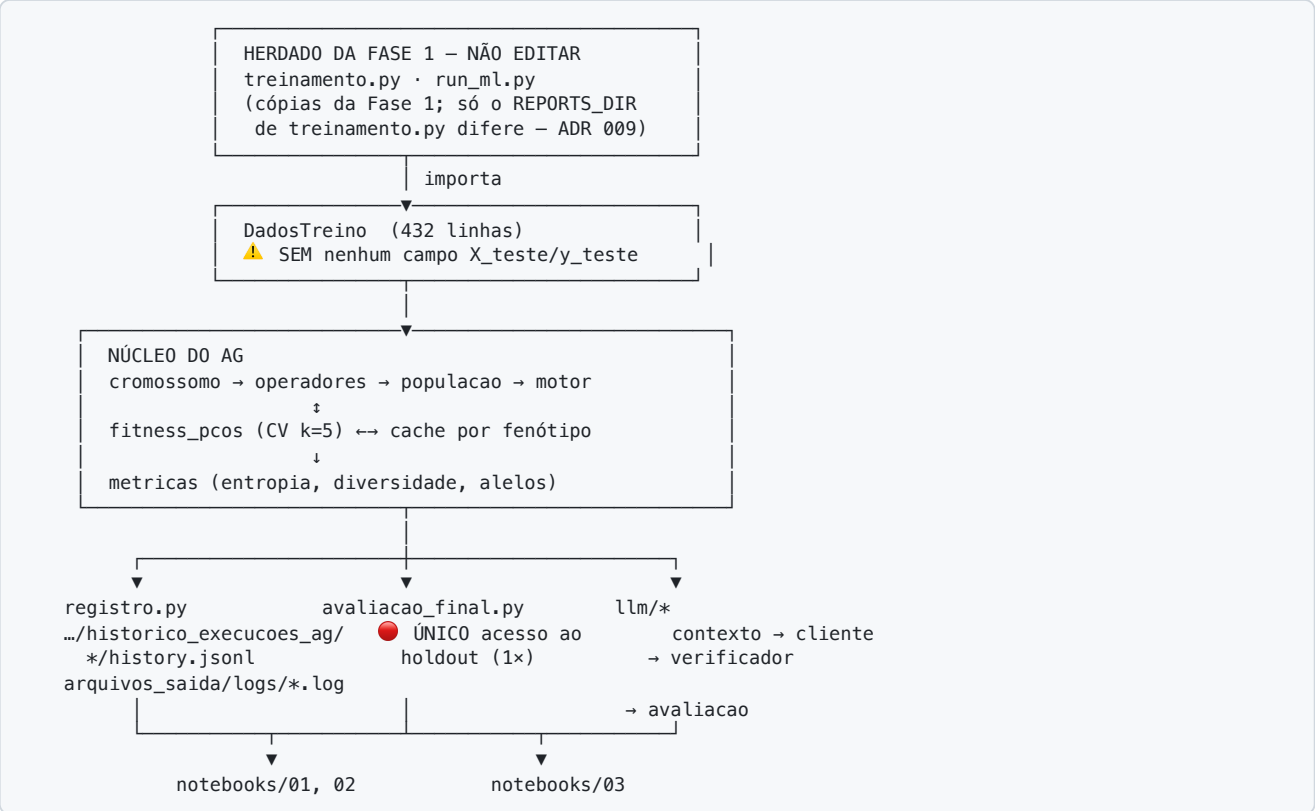
---

6. Arquitetura, monitoramento e logging

> Enunciado, requisito 2: "*Configurar recursos de escalabilidade automática para lidar com variações de demanda — implementar monitoramento e logging adequados para tracking de desempenho; documentar arquitetura e decisões de implementação. Observação: a implementação em nuvem é opcional e pode ser considerada para pontuação extra.*"

Leitura honesta deste requisito. O título diz "escalabilidade automática", que soa a autoscaling em nuvem — mas os **sub-bullets** exigem (a) monitoramento e logging e (b) documentação de arquitetura, e a observação declara a nuvem **opcional**. Como é trabalho pontual de faculdade sem implantação, atendo os dois sub-bullets de forma substantiva e trato "variação de demanda" como **variação de carga computacional do AG** — que é real, mensurável e varia **9,4× em treinos reais** entre E3 e E2. **Não deixei o requisito em branco, e não simulei uma infraestrutura que não existe.**

6.1 A arquitetura em uma página



A fronteira que organiza tudo: `DadosTreino` e `avaliacao_final.py` não se tocam (§2.5). Diagramas Mermaid de componentes e de sequência de uma geração estão em `documentacao/arquitetura.md` — o de sequência espelha o laço evolutivo descrito na abertura do §2.

Por que o AG roda como script e não dentro do notebook: execuções longas em notebook são frágeis — kernel morre, estado se perde, e o diff do `.ipynb` fica ilegível com outputs. `src/experimentos/rodar_ag.py` roda e persiste; os notebooks **leem** `arquivos_saida/arquivos_analiticos/historico_execucoes_ag/` e explicam. É também o que torna os notebooks reexecutáveis em segundos durante a gravação do vídeo.

6.2 Monitoramento e logging

- `logging` **da stdlib**, duas saídas: console (`INFO`) e arquivo (`DEBUG`), em `arquivos_saida/logs/` .
- **Log estruturado por geração** — uma linha JSON em `arquivos_saida/arquivos_analiticos/historico_execucoes_ag/<exp>_seed<n>/history.jsonl` com **18 campos**: experimento, semente, geração, melhor fitness, fitness médio, desvio padrão, entropia de família, entropia média categórica, distância euclidiana média, nº de soluções únicas, nº de genótipos únicos, frequência de `balanced` , distribuição de famílias, família do melhor, nº de avaliações, acertos de cache, e tempo total / de avaliação / de operadores. **Por que JSONL e não texto:** o notebook lê direto em DataFrame, e **todos os gráficos deste relatório saem do log, não de variáveis em memória** — a análise é reproduzível sem re-rodar o AG. Importante porque o notebook será reaberto para o relatório e para o vídeo.
- **Artefatos persistidos** por execução: `config.json` (todos os parâmetros **mais as versões de biblioteca, plataforma e processador**), `history.jsonl` , `best_individual.json` , `population_final.json` , `resumo.json` (motivo da parada, gerações, tempo, taxa de cache) e `metrics_holdout.json` . **Por que gravar o ambiente:** sem ele, um número do relatório não é rastreável. *(O plano previa um `env.json` separado; na implementação o conteúdo foi embutido em `config.json` — mesma informação, um arquivo menos.)*
- **Barra de progresso** por geração e log de uma linha por geração, com melhor/médio/desvio, entropia, únicos, família do melhor, cache e tempo.

20 execuções de AG estão gravadas em `arquivos_saida/arquivos_analiticos/historico_execucoes_ag/` , incluindo as 12 principais, as 5 da Etapa A e as 3 da Etapa C.

6.3 "Variação de demanda" → eficiência e paralelismo, com os números medidos

Alguns passos do cálculo entre uma geração e a seguinte são paralelizáveis, e vale usar processamento paralelo em problemas computacionalmente intensos.

Mecanismo	O que faz	Efeito medido
Avaliação paralela de fitness (`joblib.Parallel` , `n_jobs=-1`)	Avalia os indivíduos de uma geração em paralelo	Fitness é o gargalo absoluto (5 treinos por indivíduo) e é *embarrassingly parallel*. É o que faz E1 rodar em 32 s e E2 em 78 s
Deduplicação dentro da geração	Dois indivíduos de mesmo fenótipo na mesma geração são avaliados uma vez e o resultado é replicado	Frequente no KNN, cujo espaço de fenótipos é discreto e pequeno
Cache de fitness por fenótipo	Não reavalia indivíduo já visto	**De 6,4% para 26,9% de acerto** ao trocar a chave de genótipo para fenótipo (§7, achado nº 2). Nas execuções reportadas: 13,8% em E1 e **60,7% em E3**. **Só funciona porque o fitness é determinístico** (§2.4) — a dependência entre as duas decisões está anotada no ADR 007
Parada por estagnação	Encerra quando não há mais ganho	Economizou 60% do orçamento em E1/semente 42 (180 de 450 indivíduos) e 55% em D

> ⚠ **Armadilha de paralelismo, medida e documentada.** O `joblib` paraleliza **no nível do indivíduo**, e cada modelo roda com `n_jobs=1` . Combinar os dois níveis (`joblib` com `-1` **e** `RandomForestClassifier(n_jobs=-1)`) cria mais threads que núcleos — *oversubscription* — e fica **mais lento**. É por isso que `construir_pipeline` tem `n_jobs_modelo=1` como default, e não como descuido.

O que isso significa como "lidar com variação de demanda": a carga varia **9,4×** entre o experimento mais leve (E3, 69 treinos reais) e o mais pesado (E2, 647 treinos reais), com a mesma máquina e o mesmo código. Em tempo de parede a razão é de cerca de **4,2×** (19 s contra 83 s na reexecução de 18/10; 18,7 s contra 78,0 s na de 16/10) — e o fator de treinos é o que vale citar, porque é o único dos dois que reproduz: tempo de parede é o único campo dos artefatos que muda entre execuções idênticas. Os três mecanismos acima absorvem essa variação sem

reconfiguração: `n_jobs=-1` usa o que a máquina tem, o cache cresce de utilidade justamente quando a população fica repetitiva (E3: 60,7%), e a parada por estagnação devolve orçamento quando a busca já terminou.

6.4 Testes automatizados

A Fase 1 **não tinha** `tests/`. É requisito novo e explícito (4c), e num projeto centrado em notebooks é o mais fácil de esquecer. O núcleo do AG é código puro e determinístico — alvo ideal para pytest. **221 testes, 1.679 linhas, todos passando em 105 s.**

Arquivo	Testes	O que protege
`test_cromossomo.py`	73	Ida-e-volta codificar↔decodificar nas 5 famílias; **300 indivíduos aleatórios instanciados no scikit-learn de verdade**; `n_neighbors` sempre ímpar; pares `penalty/solver` válidos; `clamp` nas faixas; `gamma` do hotstart recalculado a partir dos dados
`test_operadores.py`	44	Crossover de pais válidos → filhos válidos **e dentro das faixas**; pais idênticos → filho idêntico; mutação respeita o espaço; prob. 0 não altera / prob. 1 altera; torneio prefere fitness maior; **população mantém tamanho constante**; elitismo preserva o melhor; **simetria de mutação entre famílias, com contraprova**
`test_fitness.py`	23	🔴 **fitness nunca acessa o conjunto de teste**; `DadosTreino` sem campo de teste; splits disjuntos; determinismo; cache acerta quando só genes latentes diferem e **não** acerta quando o fenótipo difere; hotstart de **toda** família na vizinhança da Fase 1; nenhum hotstart degenerado
`test_metricas.py`	16	**Entropia validada contra o valor analítico 1,585**; clonal → 0; uniforme → $\log_2(n)$; fenótipos idênticos contam como 1 solução
`test_motor.py`	18	Melhor fitness **nunca piora**; população constante em todas as gerações; reprodutibilidade por semente; `familia_fixa` sem escapes; parada por estagnação dispara; artefatos escritos; JSONL carrega em DataFrame; 🔴 **anti-vazamento no laço completo**
`test_llm_contexto.py`	**47**	🔴 **Campo fora da allow-list nunca entra**; campos proibidos levantam exceção; rótulo verdadeiro nunca chega ao prompt; verificador detecta número inventado e aceita arredondamento; os 3 templates existem com exemplares diversos e consistentes

Os dois testes mais importantes são os de anti-vazamento (§2.5). **Fora de escopo, declarado:** não há teste da saída da LLM por asserção de conteúdo — é não determinística, e a avaliação dela é a rubrica do §5.4.

Os 10 testes mais recentes vieram de um bug encontrado em produção, não de cobertura planejada: o verificador de termos proibidos acusava ressalvas como violações (§7, achado nº 12). Eles vêm **em par** — 5 fixam que uma ressalva negada não conta como violação, e 5 fixam que violações reais continuam detectadas. **O segundo grupo é o que importa metodologicamente:** sem ele, a correção passaria num verificador que simplesmente parou de acusar qualquer coisa. É a mesma lição do achado nº 1 (§7): um teste que só confirma a direção que você quer não é teste.

7. Desafios enfrentados e soluções implementadas

Esta seção é a mais útil do relatório e a mais desconfortável de escrever. São **doze** achados: **seis** encontrados e corrigidos durante a implementação, **cinco** na revisão final (dos quais quatro foram mantidos e estão declarados aqui em vez de silenciados), e **um** que só apareceu quando a camada de LLM rodou contra um modelo generativo real. Cada um tem a mesma estrutura: o que estava errado → qual o impacto → como foi pego → o que foi feito.

Corrigidos durante a implementação

#	Problema	Impacto	Como foi pego	Solução
1	🔴 **Hotstart do SVM gerava modelo morto (F1 = 0,000).** Valores `real_log` eram guardados crus em `HOTSTART_FASE1` e decodificados como `10**valor`: `C=1.0` virava 10 e `gamma=0,1` virava 1,26 — **58× maior** que o pretendido	**Invalidava a garantia do §2.3** de que a geração 0 ≥ Fase 1. O SVM previa tudo negativo	Auditoria manual. **Os 132 testes de então passavam com o bug presente**	`codificar_genes()` — a inversa explícita da decodificação — mais 4 testes novos. **SVM foi de F1 0,0000 para 0,8118** (Fase 1: 0,8219)

#	Problema	Impacto	Como foi pego	Solução
2	**Cache de fitness quase inoperante: 6,4% de acerto** em vez de 26,9%. A chave era o **genótipo completo**, e genes latentes derivam a cada mutação sem alterar o modelo treinado	~92 treinos desperdiçados por execução	Auditoria, medindo	Chave passou a ser `chave_fenotipo` (ADR 007)
3	**Métrica de diversidade cega à convergência prematura.** `numero_solucoes_unicas` contava genótipos	**Escondia exatamente o sinal que a métrica existe para detectar** — o §3.3 não teria resultado	Mesma causa raiz do nº 2	Passou a contar fenótipos; `numero_genotipos_unicos` ficou à parte
4	**Torneio sorteava com reposição** — podia sortear o mesmo indivíduo duas vezes e nunca comparar um segundo candidato	Pressão seletiva inconsistente, pior em populações pequenas — isto é, **justamente em E3**, o experimento sobre pressão seletiva	Teste de invariante ("torneio prefere fitness maior em >50% dos sorteios")	`rng.choice(..., replace=False)`
5	**Mutação assimétrica entre famílias** (RF sofria 2,3× mais que SVM) e **troca de família acoplada à taxa geral** (chance de manter a família por 15 gerações: 8,7% com `p=0,15`, 0,1% com `p=0,35`)	**O parâmetro dos experimentos não significava o que o §3.1 supõe que significa**: "prob_mutacao = 0,15" valia coisas diferentes para indivíduos diferentes, e o gene 0 virava passeio aleatório	Medição pós-auditoria	Normalização por nº de genes expressos + `prob_troca_familia` separada (0,05) — ADR 006
6	**O provedor `simulado` alucinava 19%** — raspava os números dos **exemplares few-shot** do próprio prompt e os reaproveitava como se fossem da paciente	O "controle de alucinação zero" não era zero. **Um instrumento de medida contaminado pelo próprio objeto não serve de controle**	Teste do próprio controle	Montagem passou a usar só o contexto formatado

A lição do achado nº 1 merece uma seção própria

Cobertura alta não é cobertura nos lugares certos. O bug mais grave do projeto sobreviveu a 132 testes porque três lacunas se somaram:

1. `test_fitness_esta_sempre_entre_zero_e_um` **aceitava 0,069 como válido** — o intervalo estava certo e a asserção era inútil;
2. `test_hotstart_reproduz_metricas_da_fase_1` verificava apenas `class_weight` no SVM, nunca `C` nem `gamma`;
3. O único teste que comparava fitness contra a Fase 1 rodava **só no Random Forest** — a única família **sem** genes `real_log`.

As três lacunas apontam na mesma direção: os testes verificavam o que era fácil verificar. As correções foram escolhidas para fechar a classe, não o caso: ciclo codificar↔decodificar nas **cinco** famílias, `gamma` recalculado a partir dos dados, hotstart de **toda** família na vizinhança da Fase 1, e guarda explícita contra modelo degenerado (F1 = 0 ou recall = 0).

Encontrados na revisão final

#	Problema	Impacto	Estado
7	**A penalidade de custo prevista (§4.3) não foi implementada.** A fórmula do fitness é a soma ponderada pura, sem `− 0,01 · tempo_normalizado`	Nenhum resultado afetado (o peso previsto, 1%, é menor que o desvio entre sementes). Mas a eficiência deixou de ser objetivo do fitness e passou a ser tratada só na camada de engenharia	**Mantido, declarado** no §2.4. Implementar invalidaria as 20 execuções gravadas, em troca de um termo que o desenho original chamava de "desempate, não objetivo"
8	**O terceiro critério de parada nunca dispara.** `piso_entropia = 0.0` e entropia nunca é negativa	A documentação anunciava três critérios ativos; só dois funcionam. E3/semente 7 ficou com `H = 0,000` em 8 de 15 gerações sem parar por isso	**Mantido, declarado** no §2.6, **com a razão pela qual manter assim é defensável**: `H = 0` na família é estado legítimo (E3/semente 7 tinha 13 de 15 fenótipos distintos com `H = 0` na geração 2). Um piso útil seria sobre fenótipos únicos, não sobre entropia
9	**O hotstart não cobre as cinco famílias.** `n_hotstart = round(tamanho × 0,1)` com atribuição cíclica: E1 cobre 3 famílias, E3 cobre 2. Na Etapa A, o motor sobrescreve o gene de família **após** a inicialização, então `A_logistic_regression`, `A_decision_tree` e `A_knn` **não têm nenhum** indivíduo com a config. da Fase 1	A garantia "geração 0 ≥ Fase 1" **valeu nas cinco famílias, mas por sorte**: os aleatórios superaram o hotstart em todas (coluna `gen0` do §4.1). Nenhum resultado é inválido. Efeito colateral: E1/E2/E3 tiveram 3/5/2 famílias cobertas — uma diferença não controlada **além** de população e mutação	**Mantido, declarado** no §2.3
10	**Cinco células erradas na documentação anterior**, por transcrição manual: entropia final de E2 (0,775 → **1,299**), entropia final de E3 (0,000 → **0,236**), cache de E1 (19% → **13,8%**), cache de E2 (15% → **4,9%**), tempo de D (33 s → **19,6 s**). E a conclusão sobre o alelo `class_weight` (§3.5) estava invertida, por ler um gráfico truncado como se fosse o fim da execução	**Era o risco mais sério para a credibilidade do trabalho**: números que não batem com os artefatos versionados ao lado deles	**Corrigido.** `README.md`, Notebook 02 e o ADR 003 foram alinhados aos valores de `analise.py`, e **todas as tabelas deste relatório são geradas a partir de `arquivos_saida/arquivos_analiticos/historico_execucoes_ag/`**. Três das cinco células erradas eram comparações **em gerações diferentes** — daí a regra de geração casada do §3.2

#	Problema	Impacto	Estado
11	**O termo de AUC do fitness deixa de medir ordenação quando o KNN tem `n_neighbors = 1`**, porque `predict_proba` só emite 0,0 e 1,0 e `roc_auc_score` colapsa em acurácia balanceada. O AG encontrou e explorou essa região	Maior regressão isolada do projeto: AUC de holdout do KNN **0,9007 → 0,7924** (§4.1). Matematicamente correto pela fórmula, e indesejado	**Mantido, declarado** no §4.1. Corrigir mudaria o espaço de busca e invalidaria as execuções gravadas
12	🔴 **O verificador de termos proibidos media o cumprimento da regra e reportava como violação.** `termo in texto`, sem tratar negação: "**não** constitui diagnóstico definitivo" — a frase que o prompt **manda** escrever — era acusada de violar a proibição de diagnosticar	**Inverteria uma conclusão do relatório.** Na primeira leitura da execução com LLM real, acusou 10 violações (9 no V1, 0 no V2), sugerindo "*o few-shot ensina o modelo a não diagnosticar*". Falso: V1 acusava mais porque é o mais verboso e escreve mais ressalvas. Após a correção, **1 ocorrência sinalizada em 45 laudos (no V1) e 0 violações reais** — a única ocorrência encaminha o diagnóstico a um especialista (§5.5)	**Corrigido.** `_esta_negado()` olha 60 caracteres à esquerda do termo, parando na fronteira de frase (uma negação na frase anterior não governa o termo). 10 testes novos: 5 de ressalva que não deve acusar e **5 de violação real que deve continuar acusando** — sem esse par, a correção passaria num verificador que simplesmente parou de acusar. Os 45 artefatos foram regravados; `test_llm_contexto.py` foi de 31 para 41 testes (hoje são 47; ver §6.4)

Lacunas menores de artefato

- `busca_aleatoria()` devolve `n_avaliacoes_efetivas` (fenótipos únicos entre os 450 sorteios), e esse campo também não é gravado. Os valores do §4.2 — 430/428/426 — foram recalculados na revisão, sorteando com a mesma semente.
- O ambiente de execução (versões e plataforma) fica dentro de `config.json`, não num `env.json` separado (§6.2).

O padrão que atravessa os doze achados

Oito dos doze (nº 1, 2, 3, 5, 9, 10, 11, 12) são o **mesmo tipo de erro**: uma representação interna confundida com o que ela representa. Genótipo confundido com fenótipo (2, 3); valor em log confundido com valor real (1); taxa nominal confundida com taxa efetiva (5); hotstart nominal confundido com hotstart expresso (9); métrica num ponto confundida com métrica no fim (10); objetivo declarado confundido com objetivo efetivo (11); e presença literal de um termo confundida com a afirmação que o termo faz (12).

Por que isso vale registrar: é a classe de erro característica de uma **codificação híbrida com genes latentes**. A decisão de projeto do §2.1 — que compra tamanho fixo, operadores triviais e reservatório de diversidade — cobra exatamente este preço: **duas camadas de representação, e todo bug mora na tradução entre elas**. Se eu refizesse

o projeto, manteria a codificação e escreveria primeiro os testes de ida-e-volta (`codificar` ↔ `decodificar`, `genótipo` → `fenótipo`), antes de qualquer operador.

---

8. Ética e limitações

8.1 Riscos de IA generativa, mapeados a este projeto

Risco	Como aparece aqui	Mitigação implementada
Privacidade (Amazon, Apple, Samsung, JP Morgan proibiram o ChatGPT internamente)	Dados de paciente num prompt enviado a terceiros	LLM **local por padrão**; allow-list de 45 campos com falha segura; campos proibidos levantam exceção; 47 testes (§5.1)
Viés de amostragem (o exemplo do modelo que "despixeliza" Obama como pessoa branca)	A base PCOS vem de **uma única clínica de fertilidade**	Limite de generalização declarado; **e a afirmação explícita de que o AG não resolve isso** — ver §8.2
Alucinação / desinformação	Um número inventado num laudo é o pior modo de falha possível	Números injetados por formatação, não gerados; verificador programático; temperatura 0,2 (§5.3)
Prompt injection	Campo textual livre chegando ao prompt	Nesta fase **não há** entrada textual livre; a superfície de risco está anotada para o Módulo 3
Responsabilidade / autonomia	O médico delegar a decisão ao texto da LLM	*Disclaimer* concatenado em código; proibição de diagnosticar no *system prompt*; `TERMOS_PROIBIDOS` detecta atos diagnósticos; a fronteira do §5.2
Antropomorfização (teoria CASA)	Texto fluente em primeira pessoa sugerindo competência clínica	Saída em seções nomeadas e impessoais, com a probabilidade do modelo explícita — formato de laudo, não de conversa

8.2 O ponto ético central: ajustar hiperparâmetros não corrige viés de amostra

E pode amplificá-lo. O fitness premia acerto na distribuição de **treino**. Se essa distribuição é enviesada — e é: uma clínica, uma população, um protocolo de coleta — então um modelo melhor ajustado é um modelo que **erra com mais confiança** nos grupos sub-representados. Nada no §2 ou no §3 endereça isso. O AG otimiza a métrica que recebe, sobre os dados que recebe.

A evidência disso está nos meus próprios dados, não num exemplo de livro. A Fase 1 comparou duas CNNs de raio-X de tórax: a **mais bem construída no papel** (BatchNorm, dropout progressivo) teve acurácia balanceada de **0,8537 na validação e 0,5598 no teste** — respondia PNEUMONIA para quase tudo —, enquanto a CNN simples fez 0,9574 e 0,8021. A causa foi **deslocamento de distribuição entre `train/` e `test/`**: viés de amostragem observado e documentado por mim, com números. **O argumento "ajustar hiperparâmetros não corrige viés de amostra" fica mais forte justamente porque eu não tentei corrigi-lo com AG** (ADR 001).

8.3 Limitações declaradas

1. **O ganho possível é estruturalmente pequeno.** Ponto de partida já bom (§1.2), 541 registros, espaço de busca de 3 a 7 genes por família. Nenhum ajuste de hiperparâmetro contorna essas três coisas, e o §4.2 mostra a consequência: três buscas independentes chegam ao mesmo ponto.
2. **O AG não se distingue de busca aleatória neste problema** (§4.2). Em qualidade de solução, empate (0,0006 de diferença em CV). O que o AG entrega de distinto é **custo menor** (~28% menos treinos) e **instrumentação do processo** (§3) — não melhor resultado.
3. **O fitness usa as mesmas 5 dobras para todos os indivíduos.** Remove ruído de amostragem como fator de confusão e viabiliza o cache, mas permite que o AG se ajuste àquela partição específica. **O preço é mensurável:** o fitness CV melhorou nas **cinco** famílias (§4.1), o score de holdout em **três**. A diferença é ajuste à partição.
4. **A rubrica qualitativa do §5.6 é por template (n = 3), não por saída** — ela escolhe entre templates e não mede variação entre pacientes. As métricas automáticas foram medidas contra `llama3.2:3b` real; o modelo é pequeno

por restrição de hardware (8 GB de RAM), o que afeta a calibração mas não a fidelidade numérica.

5. **A avaliação da LLM não é validação clínica.** Avaliador único, não-médico, n=15 pacientes. É avaliação sistemática e reprodutível no escopo de um trabalho de faculdade — dizer isso vale mais que inflar a amostra para parecer rigoroso.
6. **Quatro limitações de implementação foram mantidas** (§7, nº 7, 8, 9 e 11). Nenhuma invalida um resultado reportado; todas estão descritas com impacto.
7. **Nada de nuvem, implantação ou autoscaling real** (§6.3). O enunciado declara a nuvem opcional.

---

9. Conclusão e ponte para o Módulo 3

9.1 O que foi entregue

Requisito do enunciado	Onde	Estado
1 · AG para otimizar os modelos do Módulo 1	§2, `src/ag/`	✅ do zero, 221 testes
1a · Codificação adequada	§2.1, ADR 002	✅ híbrida, 24 genes, 6 tipos
1b · Seleção, cruzamento e mutação	§2.2, ADR 005/006	✅ despachados por tipo; **dois** operadores de seleção
1c · Fitness baseada em métricas	§2.4, ADR 003	✅ composto, com os pesos justificados clinicamente
1d · Comparar otimizado vs. original	§4.1, §4.3	✅ 5 famílias, antes/depois, + baseline aleatório (§4.2)
1e · ≥3 experimentos com configs diferentes	§3	✅ **4** configurações × 3 sementes = 12 execuções
2 · Escalabilidade para variação de demanda	§6.3	✅ paralelismo, cache, parada antecipada — com números medidos
2a · Monitoramento e logging	§6.2	✅ 18 métricas/geração em JSONL, 20 execuções gravadas
2b · Documentar arquitetura e decisões	§6.1, `documentacao/`	✅ diagramas Mermaid + **9 ADRs**
3 · Integrar LLM pré-treinada	§5.1, `src/llm/`	✅ três provedores atrás de interface fina
3a · Explicações em linguagem natural	§5.2	✅
3b · Dados numéricos → insights acionáveis	§5.2 (2, 3)	✅ relatório de experimento + resumo de coorte
3c · Base para dados textuais no Módulo 3	§5.2, §9.3	✅ interface de uma função + `prompts/` versionados
3d · Técnicas de prompt engineering	§5.3, §5.5	✅ 3 templates ancorados em achados de ICL, **comparados com LLM real**: V2 cita +49% de números em −19% de texto, e o few-shot piora a calibração (§5.5)
3e · Avaliar a qualidade das interpretações	§5.4, §5.5, §5.6	✅ métricas automáticas **medidas de verdade** sobre as 45 saídas (0% de alucinação, 100% de aderência, calibração ≤67%); **rubrica qualitativa preenchida por template, n = 3, não por saída** (§5.6)
4a · Projeto Python com ambiente virtual	`README.md`	✅ `venv` + `requirements.txt`
4b · Documentação com diagramas	`documentacao/arquitetura.md`	✅

Requisito do enunciado	Onde	Estado
4c · Testes automatizados	§6.4	✅ **221 testes**, 1.679 linhas
4d · IaC	—	🚫 fora de escopo (nuvem opcional)

O 3d fechou com a execução de 16/10/2026 contra `llama3.2:3b` local: os três templates produzem saídas distintas e a comparação tem sinal. **O 3e está atendido, com uma ressalva de resolução**: as duas formas de avaliação exigidas existem — automática no §5.5 sobre as 45 saídas, e qualitativa no §5.6 — mas a qualitativa tem resolução de template (18 julgamentos), não de saída (270). Escolher entre templates ela resolve; perfilar comportamento por paciente, não.

9.2 Os seis resultados que eu defenderia

1. **O melhor modelo da Fase 2 supera o da Fase 1 na métrica que importa clinicamente.** Regressão Logística ajustada pelo AG: recall de **0,8056** → **0,9167** sobre o Random Forest da Fase 1, com acurácia igual (0,9174) — **4 pacientes positivas a mais corretamente identificadas** entre as 36 do conjunto de teste (§4.3).
2. **O AG empata com busca aleatória em qualidade e vence em custo.** 0,0006 de diferença em CV, gastando **~28% menos treinos** de modelo (§4.2). O empate era previsto antes de rodar como cenário provável, e a causa está demonstrada: três buscas independentes convergem ao mesmo ponto.
3. **A seleção de modelo simultânea inverteu o ranking da Fase 1.** Em 9 de 12 execuções com gene 0 livre, o AG escolheu **Regressão Logística**, não Random Forest (§3.6). Nenhuma busca por família isolada revelaria isso.
4. **A convergência prematura do E3 é demonstrada, não alegada.** Em geração casada: entropia **0,000** nas três sementes, **6 de 15** fenótipos distintos, distância euclidiana → 0, cache em **60,7%** (§3.3).
5. **A escolha do torneio sobre a roleta está demonstrada por ablação.** A roleta parou por estagnação nas gerações 6, 8 e 6 contra 12 do torneio, com fitness final inferior, diversidade mais alta em todas as gerações casadas e o alelo `balanced` caindo 0,234 (§3.4, §3.5). **E a previsão que eu havia escrito tinha a observável invertida** — está mantida no texto, errada, com a explicação do porquê.
6. **Um modelo de 3B escreveu 45 laudos sem inventar um número.** 0% de alucinação numérica e 100% de aderência estrutural, medidos contra `llama3.2:3b` local (§5.5). O mérito é da **arquitetura do prompt** — números pré-escritos em código, não pedidos ao modelo — e a contraprova está no mesmo experimento: o campo que eu **deixei** o modelo derivar (a faixa de confiança) é exatamente o que nenhum template acertou acima de 67%.

9.3 Ponte para o Módulo 3

O enunciado pede "preparar a base para a futura integração com dados textuais". O que está no lugar, concretamente:

- **Ponto de extensão único.** `gerar_explicacao(contexto, template) -> str` é a única função que o resto do projeto conhece. Acrescentar RAG significa **enriquecer** `contexto` antes dessa chamada. O pipeline de ML não é tocado.
- **Prompts como artefato versionado** em `prompts/*.md`, com diff legível.
- **Allow-list já no lugar** para quando houver campo textual livre — que é quando *prompt injection* deixa de ser risco anotado e passa a ser risco ativo (§8.1).
- **As disciplinas de PLN** (tokenização, stemming, lematização, TF-IDF, Word2Vec, spaCy, NER, análise de sentimento) **não foram usadas nesta fase**, porque não há corpus textual aqui. É o Módulo 3 que as ativa: laudos, anamneses e prontuários entram como texto, e aí a allow-list, o verificador numérico e a fronteira do §5.2 passam a proteger um volume de dados muito maior. Em vez de forçar uso artificial, declaro onde cada coisa entra.
- **N8N** fica anotado como camada de orquestração não usada: num cenário de produção, uma fila de avaliações distribuída e o disparo da geração de laudos são exatamente o tipo de fluxo que a disciplina cobre.

9.4 O que eu levo deste trabalho

O resultado que mais me ensinou não foi o AG funcionando — foi **o AG empatando com busca aleatória** (§4.2) e o que foi preciso para saber disso. Sem a Etapa C, eu teria escrito "o AG melhorou os modelos", o que é verdade e é

enganoso ao mesmo tempo. A comparação custou ~1,5 min de CPU e trocou uma afirmação ambígua por uma afirmação precisa.

A segunda lição foi a do §7: **oito dos doze achados eram o mesmo erro** — confundir uma representação com o que ela representa. Foi o preço da codificação híbrida, que eu continuaria escolhendo; o que eu faria diferente é escrever os testes de ida-e-volta antes dos operadores.

E a terceira foi o §3.5 e o §3.4: **duas previsões que eu escrevi antes de rodar estavam erradas**, e eu as mantive no texto. A primeira errou o sinal da observável; a segunda errou o mecanismo. As duas me ensinaram mais do que as previsões que acertaram, e é por isso que o valor de declarar hipóteses antes é poder ser contrariado por elas — não poder exibi-las confirmadas.

A quarta chegou por último, quando a camada de LLM finalmente rodou contra um modelo real. Ela repete a segunda num contexto diferente e por isso é a que eu levo mais a sério: **três vezes neste projeto, um número agregado parecia um resultado e era um instrumento quebrado.** A métrica de diversidade contava genótipos quando o fitness depende de fenótipos (nº 3). O provedor `simulado` raspava os números dos próprios exemplares e chamava aquilo de alucinação zero (nº 6). O verificador de termos proibidos contava ressalvas como violações e quase me fez escrever que o few-shot ensina o modelo a não diagnosticar (nº 12). **Nos três casos o agregado era plausível, apontava para uma conclusão bonita, e só caiu quando eu abri o dado bruto.** A regra prática que fica: antes de escrever uma frase que começa com "os dados mostram que", abrir as linhas que produziram o número — e desconfiar especialmente quando o resultado é exatamente o que eu esperava encontrar.

Anexos — onde está cada artefato

Anexo	Onde
Templates de prompt, texto integral	**Anexo A**, abaixo — fonte em `prompts/*.md`
45 saídas brutas da LLM + planilha da rubrica	`arquivos_saida/arquivos_analiticos/avaliacao_llm/` (JSON, Markdown, CSV) — execução com `llama3.2:3b`; no CSV, o vetor de notas repete nas 15 linhas de cada template porque o julgamento é por template (§5.6)
Execução de controle (`simulado`)	`arquivos_saida/arquivos_analiticos/avaliacao_llm_simulado/` — alucinação zero por construção, calibra o verificador
Decisões arquiteturais (9 ADRs)	`documentacao/decisoes/001` a `009`
Diagramas de componentes e de sequência	`documentacao/arquitetura.md`
Artefatos das 20 execuções de AG	`arquivos_saida/arquivos_analiticos/historico_execucoes_ag/<experimento>_seed<n>/`
Logs estruturados	`arquivos_saida/logs/*.log`, `arquivos_saida/arquivos_analiticos/historico_execucoes_ag/*/history.jsonl`
Notebooks narrados	`notebooks/01`, `02`, `03`

Como reproduzir tudo:

```
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
python -m src.run_ml                                        # baseline da Fase 1 (~10 s)
python -m pytest tests/ -q                                  # 221 testes (~105 s)
python -m src.experimentos.rodar_ag --experimento todos     # E1,E2,E3,D × 3 sementes (~3 min)
python -m src.experimentos.rodar_ag --etapa-a               # AG por família (~2 min)
python -m src.experimentos.rodar_ag --etapa-c               # busca aleatória (~1,5 min)
jupyter lab                                                 # os 3 notebooks leem arquivos_saida/arquivos_analiticos/
```

Anexo A — Os três templates de prompt, texto integral

Os três arquivos vivem em `prompts/` e são lidos por `carregar_template()`, que divide no marcador `---USUARIO---` em (system prompt, molde do usuário). **Por que em arquivo e não em string no código:** prompt é artefato experimental, e precisa de diff legível no Git para mostrar a evolução das versões (§5.2). O enunciado pede "os prompts utilizados" no relatório — por isso o texto integral está aqui, e não apenas referenciado.

> ⚠️ Os templates abaixo são exatamente o que foi enviado ao `llama3.2:3b` na execução reportada no §5.5. O provedor `simulado`, mantido só como controle, **ignora o template** e produz a mesma saída para os três.

A.1 — V1: zero-shot com papel e estrutura

`prompts/v1_zero_shot.md`

```
Você é um assistente que apoia profissionais de saúde na **interpretação** de saídas de um
modelo estatístico de triagem para Síndrome dos Ovários Policísticos (SOP).

REGRAS ABSOLUTAS:

1. Você **não** emite diagnóstico. O modelo produziu uma estimativa estatística; seu papel é
   traduzi-la para linguagem clara. A decisão clínica é do profissional de saúde.
2. Use **exclusivamente** os números fornecidos no contexto. Nunca calcule, estime ou invente
   valores. Se um dado não estiver no contexto, não o mencione.
3. Declare sempre a incerteza e as limitações do modelo.
4. Escreva em português do Brasil, para leitor com formação médica. Seja direto; evite jargão
   de aprendizado de máquina.
5. Produza exatamente estas quatro seções, com estes títulos:
   - `## Resumo da indicação do modelo`
   - `## Variáveis que mais pesaram`
   - `## Grau de confiança e limitações`
   - `## Próximo passo sugerido`

---USUARIO---

Interprete a saída do modelo para esta paciente.

DADOS (use somente estes números):
{dados}

CONTEXTO COMPLETO (JSON):
{contexto}
```

A.2 — V2: few-shot (V1 + 3 exemplares)

`prompts/v2_few_shot.md`

```
Você é um assistente que apoia profissionais de saúde na **interpretação** de saídas de um
modelo estatístico de triagem para Síndrome dos Ovários Policísticos (SOP).

REGRAS ABSOLUTAS:

1. Você **não** emite diagnóstico. O modelo produziu uma estimativa estatística; seu papel é
   traduzi-la para linguagem clara. A decisão clínica é do profissional de saúde.
2. Use **exclusivamente** os números fornecidos no contexto. Nunca calcule, estime ou invente
   valores. Se um dado não estiver no contexto, não o mencione.
3. Declare sempre a incerteza e as limitações do modelo.
4. Escreva em português do Brasil, para leitor com formação médica. Seja direto; evite jargão
   de aprendizado de máquina.
5. Produza exatamente estas quatro seções, com estes títulos:
   - `## Resumo da indicação do modelo`
   - `## Variáveis que mais pesaram`
   - `## Grau de confiança e limitações`
   - `## Próximo passo sugerido`

---USUARIO---

Veja três exemplos de interpretações adequadas. Note que os três cobrem situações
**distintas** — alta probabilidade, baixa probabilidade e caso próximo do limiar — mas seguem
**exatamente** a mesma estrutura.

### EXEMPLO 1 — alta probabilidade

DADOS:
- Probabilidade estimada pelo modelo: 91,2%
- Classificação do modelo: indicativo de SOP
- Variáveis de maior peso no modelo, com os valores desta paciente:
    - Follicle No. (R): 18 (peso no modelo: 0.2140)
    - Follicle No. (L): 16 (peso no modelo: 0.1980)
    - Cycle(R/I): 4 (peso no modelo: 0.0710)

INTERPRETAÇÃO:
## Resumo da indicação do modelo
O modelo classificou o caso como indicativo de SOP, com probabilidade estimada de 91,2%.

## Variáveis que mais pesaram
A contagem de folículos foi o fator de maior peso: 18 à direita e 16 à esquerda. A
irregularidade do ciclo (valor 4) também contribuiu, com peso menor.

## Grau de confiança e limitações
A probabilidade de 91,2% está na faixa alta de confiança do modelo. Ainda assim, trata-se de
estimativa estatística derivada de dados de uma única clínica de fertilidade; o modelo não
considera histórico clínico completo nem avaliação presencial.

## Próximo passo sugerido
Correlacionar a contagem folicular com os critérios diagnósticos vigentes e com a avaliação
clínica antes de qualquer conduta.

### EXEMPLO 2 — baixa probabilidade

DADOS:
- Probabilidade estimada pelo modelo: 8,4%
- Classificação do modelo: sem indicativo de SOP
- Variáveis de maior peso no modelo, com os valores desta paciente:
    - Follicle No. (R): 5 (peso no modelo: 0.2140)
    - Cycle(R/I): 2 (peso no modelo: 0.0710)
    - AMH(ng/mL): 1.8 (peso no modelo: 0.0520)

INTERPRETAÇÃO:
## Resumo da indicação do modelo
O modelo classificou o caso como sem indicativo de SOP, com probabilidade estimada de 8,4%.

## Variáveis que mais pesaram
A contagem folicular à direita (5) ficou na faixa que o modelo associa a casos sem
indicativo. O ciclo regular (valor 2) e o AMH de 1,8 ng/mL reforçaram essa classificação.

## Grau de confiança e limitações
A probabilidade de 8,4% indica baixa suspeita segundo o modelo. Um resultado negativo não
exclui a condição: o modelo tem sensibilidade limitada e foi treinado em população específica.

## Próximo passo sugerido
Manter acompanhamento de rotina. Reavaliar caso surjam sinais clínicos não capturados pelas
variáveis do modelo.
```

```
### EXEMPLO 3 — caso próximo do limiar

DADOS:
- Probabilidade estimada pelo modelo: 52,7%
- Classificação do modelo: indicativo de SOP
- Variáveis de maior peso no modelo, com os valores desta paciente:
    - Follicle No. (R): 11 (peso no modelo: 0.2140)
    - Follicle No. (L): 8 (peso no modelo: 0.1980)
    - Weight gain(Y/N): 1 (peso no modelo: 0.0450)

INTERPRETAÇÃO:
## Resumo da indicação do modelo
O modelo classificou o caso como indicativo de SOP, mas com probabilidade de 52,7% — muito
próxima do limiar de decisão.

## Variáveis que mais pesaram
A contagem folicular (11 à direita, 8 à esquerda) ficou em faixa intermediária, o que explica
a indicação pouco definida. O relato de ganho de peso contribuiu com peso baixo.

## Grau de confiança e limitações
**Este é um caso de baixa confiança.** A 52,7%, a classificação do modelo é praticamente um
empate e não deve orientar conduta isoladamente.

## Próximo passo sugerido
Priorizar avaliação clínica e exames complementares: a saída do modelo, neste caso, não
agrega informação decisiva.

---

Agora interprete a saída do modelo para esta paciente, seguindo exatamente a mesma estrutura.

DADOS (use somente estes números):
{dados}

CONTEXTO COMPLETO (JSON):
{contexto}
```

A.3 — V3: few-shot + raciocínio estruturado

`prompts/v3_few_shot_estruturado.md`

```
Você é um assistente que apoia profissionais de saúde na **interpretação** de saídas de um
modelo estatístico de triagem para Síndrome dos Ovários Policísticos (SOP).

REGRAS ABSOLUTAS:

1. Você **não** emite diagnóstico. O modelo produziu uma estimativa estatística; seu papel é
   traduzi-la para linguagem clara. A decisão clínica é do profissional de saúde.
2. Use **exclusivamente** os números fornecidos no contexto. Nunca calcule, estime ou invente
   valores. Se um dado não estiver no contexto, não o mencione.
3. **Não explique mecanismo fisiopatológico.** Você não tem como saber por que uma variável
   se associa à condição; você sabe apenas que o modelo atribuiu peso a ela. Diga "o modelo
   atribuiu peso a X", nunca "X causa Y".
4. Declare sempre a incerteza e as limitações do modelo.
5. Escreva em português do Brasil, para leitor com formação médica. Seja direto; evite jargão
   de aprendizado de máquina.
6. Produza exatamente estas quatro seções, com estes títulos:
   - `## Resumo da indicação do modelo`
   - `## Variáveis que mais pesaram`
   - `## Grau de confiança e limitações`
   - `## Próximo passo sugerido`

PERCURSO DE RACIOCÍNIO OBRIGATÓRIO — siga nesta ordem, sem pular etapas:

   (a) Qual foi a classificação e qual a probabilidade? → seção 1
   (b) Quais variáveis tiveram maior peso, e que valor esta paciente apresentou em cada? → seção 2
   (c) A probabilidade está em faixa alta (>70%), intermediária (30–70%) ou baixa (<30%)?
       Casos entre 40% e 60% devem ser explicitamente sinalizados como **baixa confiança**. → seção 3
   (d) O que este resultado **não** autoriza a concluir? → seção 3
   (e) Qual o próximo passo proporcional ao grau de confiança? → seção 4

---USUARIO---

Veja três exemplos de interpretações adequadas. Note que os três cobrem situações
**distintas** — alta probabilidade, baixa probabilidade e caso próximo do limiar — mas seguem
**exatamente** a mesma estrutura e o mesmo percurso de raciocínio.

### EXEMPLO 1 — alta probabilidade

DADOS:
- Probabilidade estimada pelo modelo: 91,2%
- Classificação do modelo: indicativo de SOP
- Variáveis de maior peso no modelo, com os valores desta paciente:
    - Follicle No. (R): 18 (peso no modelo: 0.2140)
    - Follicle No. (L): 16 (peso no modelo: 0.1980)
    - Cycle(R/I): 4 (peso no modelo: 0.0710)

INTERPRETAÇÃO:
## Resumo da indicação do modelo
O modelo classificou o caso como indicativo de SOP, com probabilidade estimada de 91,2% —
faixa alta de confiança.

## Variáveis que mais pesaram
O modelo atribuiu maior peso à contagem de folículos (0,2140 à direita e 0,1980 à esquerda);
esta paciente apresentou 18 e 16, respectivamente. A irregularidade do ciclo (valor 4)
recebeu peso menor, 0,0710.

## Grau de confiança e limitações
A 91,2%, a indicação está na faixa alta. O resultado **não** autoriza concluir diagnóstico:
é estimativa estatística de um modelo treinado em dados de uma única clínica de fertilidade,
que não considera histórico clínico completo, exames de imagem nem avaliação presencial.

## Próximo passo sugerido
Correlacionar a contagem folicular com os critérios diagnósticos vigentes e com a avaliação
clínica antes de qualquer conduta.

### EXEMPLO 2 — baixa probabilidade

DADOS:
- Probabilidade estimada pelo modelo: 8,4%
- Classificação do modelo: sem indicativo de SOP
- Variáveis de maior peso no modelo, com os valores desta paciente:
    - Follicle No. (R): 5 (peso no modelo: 0.2140)
    - Cycle(R/I): 2 (peso no modelo: 0.0710)
    - AMH(ng/mL): 1.8 (peso no modelo: 0.0520)

INTERPRETAÇÃO:
## Resumo da indicação do modelo
```

```
O modelo classificou o caso como sem indicativo de SOP, com probabilidade estimada de 8,4% —
faixa baixa.

## Variáveis que mais pesaram
A contagem folicular à direita, variável de maior peso (0,2140), foi de 5 nesta paciente. O
ciclo (valor 2, peso 0,0710) e o AMH de 1,8 ng/mL (peso 0,0520) também contribuíram para a
classificação negativa.

## Grau de confiança e limitações
A 8,4%, a suspeita é baixa segundo o modelo. O resultado **não** exclui a condição: a
sensibilidade do modelo é limitada e ele foi treinado em população específica, não
representativa da população geral.

## Próximo passo sugerido
Manter acompanhamento de rotina. Reavaliar caso surjam sinais clínicos não capturados pelas
variáveis do modelo.

### EXEMPLO 3 — caso próximo do limiar

DADOS:
- Probabilidade estimada pelo modelo: 52,7%
- Classificação do modelo: indicativo de SOP
- Variáveis de maior peso no modelo, com os valores desta paciente:
    - Follicle No. (R): 11 (peso no modelo: 0.2140)
    - Follicle No. (L): 8 (peso no modelo: 0.1980)
    - Weight gain(Y/N): 1 (peso no modelo: 0.0450)

INTERPRETAÇÃO:
## Resumo da indicação do modelo
O modelo classificou o caso como indicativo de SOP, com probabilidade de 52,7% — faixa
intermediária, muito próxima do limiar de decisão.

## Variáveis que mais pesaram
O modelo atribuiu maior peso à contagem folicular (0,2140 e 0,1980); esta paciente apresentou
11 à direita e 8 à esquerda, valores intermediários. O relato de ganho de peso recebeu peso
baixo, 0,0450.

## Grau de confiança e limitações
**Este é um caso de baixa confiança.** A 52,7%, a classificação é praticamente um empate
estatístico e **não** deve orientar conduta isoladamente. O modelo não dispõe de informação
suficiente para discriminar este caso.

## Próximo passo sugerido
Priorizar avaliação clínica e exames complementares: a saída do modelo, neste caso, não
agrega informação decisiva.

---

Agora interprete a saída do modelo para esta paciente, seguindo exatamente a mesma estrutura e
o percurso de raciocínio obrigatório.

DADOS (use somente estes números):
{dados}

CONTEXTO COMPLETO (JSON):
{contexto}
```